



***Design and Development of an AI-Powered Cloud-Based Public Notice
Management System for Nepal***

By

Ashok Bhattarai

NP069811

NP3F2509IT

A report submitted in partially fulfillment of the requirements for the degree of
B.Sc. (Hons) Information Technology Specialism in Cloud Engineering
at Asia Pacific University of Technology and Innovation

Supervised by Dr. Laxman Mandal

2nd Marker: Manish Kumar Tamang

2026

Acknowledgement

The support and motivation that many have offered me, I cannot overestimate. My deepest thanks extend to my project supervisor, who has been of great help in guidance, offering constructive feedback, and support throughout the research. The overall mentorship directed and steered the project well.

I am very grateful to the academic staff of the School of Computing, Asia Pacific University of Technology and Innovation. They infused me with knowledge, support, and very practical insights during lectures, consultation, and assessments, which formed a sound base for this work in artificial intelligence, cloud computing, and software engineering.

I would like to express my gratitude to my colleagues and peers for their continuous and ever-decisive encouragement; without their technical exchanges and constantly pushing critical suggestions, the research work would not have turned out as meaningful as it has.

Special Acknowledgment to the open-source developer community that provided tools, frameworks, and documentation that were used in this project: Hugging Face, LangChain, Scrapy, ChromaDB, and, of course, the larger Python and JavaScript ecosystems.

Finally, to my deepest feeling, I owe my family for staying there with constant support, patience, and encouragement throughout this academic journey. My motivation and strength hails from their constant belief in me.

Yours Sincerely

Ashok Bhattarai (NP069811)

Abstract

That information systems in Nepal are pretty much in a shambles is not a fallacy. Public notices, vacancies, and tenders, including exam dates and policy updates, are posted across dozens of independent websites without any central point of access. This not only makes it hard for every other person but even more problematic for students, persons looking for jobs, and those living in rural resources who scarcely get the information in time and with authenticity.

This investigation report is on AI-Powered Cloud-Based Public Notice Management System. It is a web platform that collects the public notices from a few selected official Nepalese government portals, classifies them, and summarizes them in a searchable form. Scrapy and Selenium are the web scraping tools; OCR is by Tesseract. The first is for classification, and then summarization of text using _ models from Hugging Face. Two of its integrations are LangChain and ChromaDB, developed for document-based question answering.

The platform shall provide a single interface to the users with AI-generated summaries and improved access to official notices. It will be scalable and available with cloud deployment using Amazon Web Services. This report elucidates the research background, literature review, system design, methodology, and primary results in confirmation of the technical feasibility and practical relevance of the proposed solution.

Keywords: *public notice management, artificial intelligence, cloud computing, web scraping, retrieval-augmented generation, Nepal e-governance*

Table of Contents

Chapter 1: Introduction.....	1
1.1 Introduction	1
1.2 Problem Background	2
1.3 Project Aim.....	4
1.4 Objectives.....	4
1.5 Scope.....	5
1.6 Potential Benefits.....	6
Tangible Benefits.....	6
Intangible Benefits.....	7
Target User.....	7
Functionality (Deliverables).....	8
1.7 Overview of the Investigation Report.....	8
1.8 Project Plan.....	9
Chapter 2: Literature Review	11
2.1 Domain Research.....	11
2.1.1 E-Governance and Public Information Systems.....	11
2.1.2 Natural Language Processing for Classification and Summarisation	12
2.1.3 Retrieval-Augmented Generation	13
2.1.4 Web Scraping Technologies	14
2.1.5 Cloud Computing and Deployment Architecture	15
2.2 Similar Systems and Works	16
2.2.1 Government Notice Aggregation Platforms	16

2.2.2 AI-Powered Document Processing Platforms	16
2.2.3 Comparative Analysis and Gap Identification.....	17
2.3 Technical Research.....	17
2.3.1 System Requirements Analysis	18
2.3.2 Hardware Requirements	18
2.3.3 Software Selection and Justification	19
Chapter 3: Methodology	20
3.1 System Development Methodology.....	20
3.1.1 Development Phases	22
3.1.2 System Architecture	25
3.1.3 Data Flow Diagram.....	30
3.2 Data Gathering Design.....	31
3.2.1 Survey Design.....	31
3.2.2 Interview Design.....	32
3.2.3 Secondary Data Collection via Web Scraping.....	32
3.3 Analysis.....	33
3.3.1 Quantitative Analysis Approach.....	34
3.3.2 Qualitative Analysis Approach.....	34
3.3.3 Preliminary User Requirements.....	35
Chapter 4: Conclusion	38
4.1 Achievement of Investigation Phase Objectives	38
4.2 Research Gaps and Areas for Further Development	39
4.3 Reflection on the Investigation Phase.....	41

References	43
Appendices.....	51
Appendix A: Project Submission Form	51
Appendix B: Project Proposal Form.....	59
Appendix C: Ethics Forms.....	68
Appendix D: Supervisor Meeting Log Sheets	73
Appendix E: Gantt Chart	78
Appendix F: Respondent Demographic Profile	79

Table of Figures

Figure 1 Agile Methodology	21
Figure 2 System architecture diagram	26
Figure 3 Data flow diagram	30

Chapter 1: Introduction

1.1 Introduction

Public information systems are one of the cornerstones for enhancing civic engagement and informed choice in democratic societies. In a week, governments generate a mix of official communications that run from time tables of examinations and announcements of job vacancies to tender notices, policy changes, and administrative directives. These communications are of legal consequence and sometimes practical importance to millions of citizens and organizations needing timely access to official information so they can make informed decisions about issues related to education, employment, procurement, and compliance (Nchaga, 2025).

The government of Nepal issues loads of official notifications through ministries, commissions and regulatory bodies on their respective websites. But the problem is that there is no standard format for publication, no central repository, and absolutely no search mechanism that could be of any help to a citizen in tracing the correct information he or she is looking for. This means that a student who is waiting for examination intimation from the Public Service Commission, a contractor monitoring tender announcements from the Ministry of Finance, or a job seeker peeping into vacancy notifications from several other government departments would have to keep on personally visiting different websites at repetitive intervals over a few days or a couple of weeks to get updates (Shrestha et al., 2023).

The imperfect and inadequate model of public information diffusion is a systemic problem for civic access, as well as government transparency. It is further compounded by the fact that most of the government portals are technologically inconsistent; many are seen publishing their gazettes as scanned PDF files or embedded images rather than as machine-readable text, hence not even allowing for basic keyword search text. This has produced a

public information environment akin to dissonance with citizens that are digitally minded and absolutely incongruent with international best practices of e-governance (phuyal, 2024).

This document concerns the preliminary design and progress report of the requirements of my Final Year Project: AI-Powered Cloud-Based Public Notice Management System for Nepal. The system automatically collects, classifies, summarizes, and retrieves public notices through a cloud-hosted, user-facing web platform that is enhanced with natural language processing capabilities and retrieval-augmented generation. This first chapter will introduce the project, problematic background, state the aim and objectives of the project, define the scope, and outline potential benefits. It will further present an overview of the structure of the report (Nirman, 2025).

1.2 Problem Background

The primary issue is the lack of accessibility. The fragmentation and chronic inaccessibility of public information within the digital administrative landscape of Nepal are highly problematic, to say the least. In the present scenario, official announcements of bodies within the Nepali government get published one by one on dozens of independent government websites. They all follow unique design conventions and content organization strategies, each one having a different publication schedule. It also lacks a standard metadata schema for notices, no cross-portal search capability, category-based filtering mechanism, or subscription/alert service that allows citizens to receive notification for the types of notices that may be relevant to them (Rayamajhi, 2026).

The major part of government notices in Nepal are released in forms that cannot be searched and processed inherently. Although most of the gazetted notices are in the form of scanned PDFs, here, the text is rasterized as an image and not recognized as machine-readable characters. This makes it not possible to have the content indexed by search engines,

extracted by text processing tools, or read with a screen reader. Consequently, such information is virtually not accessible to users who cannot manually open and read an entire document. The problem becomes more serious for low-bandwidth users, as implications to the cost of data and loading time for repeatedly downloading large files in a PDF format from various web portals really become a burden (Elen, 2023).

It's a problem that also has a geographic and equity dimension. Urban dwellers have relatively easy access to broadband and high digital literacy that helps them switch between various government portals and stay updated. But citizens in rural and semi-urban areas are largely burdened with the quest for the same information due to intermittent internet connectivity and high data costs relative to their income levels. Structural weaknesses in access to equitable civic opportunities breed on information asymmetry within Nepal's e-governance framework (Elen, 2023).

Currently, there is no government or non-governmental mechanism in Nepal that pulls together and organizes separate notifications from multiple government sources, classifies or summarizes these with AI, or provides a document-based question-answering service for official notifications. No private company engages in a daily news aggregation service that comprehensively covers administrative notifications together with verification, structurally formulated to meet information retrieval requirements for students, job seekers, or compliance professionals. This gap in the country's public information ecosystem is what the project tries to fill (Miah, 2024).

Developing such a system is technically very challenging. The technical architecture, the structure of HTML, and updating mechanisms are very different for government websites in Nepal. In view of this diversified landscape, reliable scrapers should be developed with adaptive strategies for scraping static HTML content as well as Javascript-rendered pages. The need to extract text from scanned documents entails strong OCR pipelines with

validation of quality. The integration of several AI components related to classification, summarization, and document interaction within a cloud-hosted architecture calls for a cautious system design and modular implementation (Jagadish et al., 2023).

1.3 Project Aim

The purpose of this research project is to develop a web-based application that uses AI technologies to automate the process of collecting, categorising, summarising, and delivering intelligent access to notices available publicly on certain Nepalese government sites so that the citizens can have easy access and understand these announcements (Bagale, 2025).

1.4 Objectives

The objectives listed below have been established in order to operationalize the project's aim and to guide the development process:

1. Examine the present situation related to the publication of public notices in Nepal by sources like core government portals. Common categories of notices, technical problems of data extraction, and citizen access barriers will be explained by target user groups.
2. To design and setup web scraping and data processing pipeline to automatically fetch, extract, store, and normalize all public notices from a minimum of five selected government web sites, having mechanisms to deal with both static and JavaScript-rendered pages, and PDF-based or image-embedded notices using Optical Character Recognition.
3. To integrate NLP models into a predefined format for the automatic classification of retrieved notices and generate concise, plain language summaries. Develop a retrieval-augmented generation module that allows users to upload documents and receive grounded natural language answers to their queries.

4. The purpose of this activity is to develop, deploy, and evaluate a citizen-facing web application, which is cloud-hosted and offers unified search, filter, browse, and alert functionality across aggregated government notices. It compares the usability, accuracy, and performance of the system against the defined success criteria as provided.

1.5 Scope

This project's scope is defined by those specifications and constraints: the system will operate on a proof of concept based on a minimum of five and a maximum of six identified key Nepalese government websites out of the ones selected as primary data sources. For the notice publication frequency, civic relevance, and technical accessibility, among others, of a number of sites that include portals like the Public Service Commission, the Ministry of Finance, and the Judicial Service Commission, among others (Wrange et al., 2023).

Categorization of notices will be carried out under the following predefined categories: job postings, exam notifications, tender and procurement notifications, policy news, and other administrative notices. All machine learning methods, such as categorization, summarization, and generation augmented by retrieval, will be applied on notices that are posted using English and Nepali-transliterated text. Devanagari NLP techniques are not within the scope of this research and need to be explored further in the future (Bista, 2023).

For the retrieval-augmented generation component, the document upload capability will be enabled using PDF and images, but the real-time voice interface, cross-referencing of multiple documents in one request, and answer verification with external sources will not be supported. It will be developed as a responsive web app that will function in regular desktop and mobile browsers; developing a native mobile app will be out of scope for the project (Shakya et al., 2023).

Deployment will happen in the AWS environment for backend and database services and in the Vercel environment for the frontend. Development will take place in the local environment that meets the minimum requirement mentioned in the technical research portion. For testing purposes, around 200-300 notifications will be loaded into the test database that is obtained from the target government websites, and they will cover all five notification types (Al-Zoubi, 2009).

1.6 Potential Benefits

Tangible Benefits

- The citizens will be able to view all public notices issued by different governmental authorities in one place without having to check different sites, saving them a lot of time in the process.
- Plain language summaries of the AI-generated content will drastically cut down on the workload of having to read lengthy government notices, allowing for an immediate determination of whether the document applies to one's interests.
- A searchable and filterable notice database will allow for quick searching and locating of particular notice types, categories, keywords, dates, and organizations. This will greatly improve the efficiency of obtaining information through searching rather than manually browsing the portals.
- The optional generation-augmented retrieval module will enable the user to submit either a PDF or an image file, and then pose targeted natural language queries, which will yield grounded answers extracted from the content of the document.
- Subscription alerts can be configured so that users get automatic alerts when there are new alerts that fall under the category they have subscribed to or have entered as keywords.

Intangible Benefits

- It will seek to bridge the information gap between urban and rural areas so that participation in civil affairs is as equal as possible, especially for those without sufficient digital means.
- The increased availability, transparency, and access to government information may enhance public awareness and participation in governmental activities.
- Through a robust information provision system that is precise and logically organized, there will be an increase in public trust in e-government services and adoption of digital platforms to access civic information in Nepal.
- Those students, workers, and professionals who require up-to-date information on exams' schedule, vacancies, and tenders will get great benefits from using the capabilities of artificial intelligence for searching and notification services on the platform.

Target User

The AI-Powered Cloud-Based Public Notice Management System is aimed at users as enlisted below:

1. **Students** - The target will be chiefly university and school students who access several things, like exam schedules, result announcements, scholarship notices, from among those published by Public Service Commission and Tribhuvan University Office of Controller of Examinations.

2. **Job Seekers**- This target group is made up of everyone who is actually looking for a job and needs to know about vacancies when they are announced, or about civil service recruitments in the ministries and commissions of the government.
3. **Working Professionals and Contractors** - This will include all those professionals, working in both public and private sectors, who are interested in keeping a track of tender calls, procurement notices, and policy directives that are sector-specific or compliance-bound.
4. **Administrators** - System administrators are to be responsible for scraping pipelines and moderating ingested notices, and classifying labels to use in monitoring system health and visualization of performance.

Functionality (Deliverables)

The AI-Powered Public Notice Management System in Cloud enables the following functionalities to the general users:

1. Ability to sign in and manage their personal accounts
2. Can perform searches on public notices via keywords, categories, date ranges, and source organizations
3. Can search, filter, and view aggregated notices from multiple government portals within one unified interface
4. View an AI-generated plain language summary beside each notice listing
5. Ability to set up or manage subscriptions to alerts by keyword or category, so they are alerted when new, relevant notices are published.

1.7 Overview of the Investigation Report

The four chapters of this Investigation Report are structured as follows: Chapter 1 provides the project introductory background, problem, aim, objectives, scope, and benefits, while Chapter 2 details an extensive literature review in the domain of research in e-governance and public information systems, finally moving to natural language processing for text classification and summarization, retrieval-augmented generation, and cloud computing architectures. Further, it discusses a comparative review among similar systems and technical research reviews justifying the technology and tools used. Chapter 3 depicts the methodology opted for research during the project. It includes the Agile system development approach with eight identified phases, primary data gathering design through user surveys, semi-structured interviews, and strategies for data analysis that will lead to preliminary user requirements and the full suite design diagrams of the system. Chapter 4 concludes the Investigation Report by summarizing the accomplishments during the investigation phase, discussing the research gaps identified and those areas that need further development, and making a forward-looking statement in order to position the project for the implementation stage (Giri et al., 2025).

1.8 Project Plan

The Agile methodology was adopted to work on this project; it was divided into eight phases that ran consecutively but iteratively, spread over two semesters. The Key Deliverable in Semester 1 shall be an Investigation Report. In the first phase, project planning and architectural design shall be completed, system requirements finalized, development environment set up, and scaffolding of core frameworks. The second is to implement the web scraping module over all target government websites. The third phase will be to set up a cloud pipeline for storing and processing the data. The fourth phase incorporates the models for AI classification and summarisation. In this phase, a user-facing frontend is developed. The sixth

phase deals with the retrieval-augmented generation module to manage the document interaction. The seventh phase involves testing in all directions and deployment over cloud infrastructure, while the eighth phase generates full project documentation (Veturi et al., 2024). For these phases, the project timeline is detailed in the Gantt chart in Appendix E.

Chapter 2: Literature Review

2.1 Domain Research

2.1.1 *E-Governance and Public Information Systems*

“E-Governance can be defined as the use of Information Technology in delivering government services to improve the performance of government processes and to foster transparency and civic engagement.” The underlying theory of E-governance is based on Public Administration, Information Systems Theory, Political Science, and has been well discussed by researchers exploring factors that determine whether digital governance projects succeed in creating public value or not (Efthymiou, 2025).

Janssen, Charalabidis, and Zuiderwijk have carried out an extensive analysis on various cases of open data projects in different governmental scenarios, and in all cases, a persistent gap between the technical accessibility of government information and its usability has been found. This has led researchers to conclude that the use of open government data does not meet the expectations due to poor metadata practices, irregular publication practices, lack of design orientation towards the users, and lack of efforts to involve the citizens and make them literate with respect to the data. These deficiencies can be observed in the notice generation scenario in Nepal (Janssen et al., 2012).

The researchers Wirtz, Weyerer, and Geyer studied the implementation of AI in the public sector and found that when implemented properly, AI can have many benefits in making public information services more accessible, efficient, and responsive. The research identified text classification, information extraction, and natural language generation as some of the AI capabilities that are practical to implement in public sectors due to their similar functionality to the intended functionality of the proposed public information service.

Nevertheless, the research notes that proper governance, auditing, and oversight processes are required in order for the AI system to function accurately and objectively (Wirtz et al., 2018).

The other framework for developing e-governance in South Asia is provided by UNESCO and UNDP, which emphasizes the need for inclusive design in e-governance portals because portals that are designed only for digitally literate and urban dwellers often overlook rural users, those who have low levels of literacy, and those with few technological capabilities. The proposed portal recognizes these factors, focusing on designing an accessible and usable portal that provides a minimalistic design that focuses on functionality (Kader et al., 2025).

2.1.2 Natural Language Processing for Classification and Summarisation

Natural language processing is an AI subdiscipline that focuses on making computers capable of interpreting, generating, and modifying natural languages. The last decade has seen a revolution in NLP abilities owing to the emergence and popularity of deep learning models based on the transformer architecture, specifically the attention model pioneered by Vaswani et al. in the groundbreaking work “Attention Is All You Need,” which laid down the groundwork for models like BERT, GPT, T5, and other variants (Omweri, 2025).

The effectiveness of deep learning text classification algorithms has been shown to be superior to those of previous solutions based on recurrent neural networks, convolutions, and feature engineering by Minaee et al., who have reviewed state-of-the-art text classification methods based on deep learning, revealing the superiority of the transformer architecture for all classification benchmarks studied. Transformer models pre-trained on domain-specific datasets can be especially effective, and the existence of such models as provided by the Hugging Face Transformers package greatly reduces the complexity of implementing efficient classification workflows (Ahmed et al., 2025).

Two different methodologies exist in text summarization. In extractive text summarization, the best representative sentences/phrases of the original text are selected without using any new language in the process, whereas in abstractive text summarization, the summarizing model creates new text to capture the essence of the input text. Abstractive summarization models with transformers architecture, including BART and T5 models, are capable of creating coherent text with respect to semantics and structure from any formal document such as official notices. For the proposed system, abstractive text summarization is the methodology of choice as official notices often have formal preambles or procedure-specific language which, if used in the summary directly, may not be meaningful for citizens (Varade et al., 2021).

The Hugging Face Transformer library offers an extensive collection of pre-trained models as well as fine-tuned models that are well documented for the deployment process. The standard model interface provided by the Transformer library coupled with its compatibility with the rest of the Python scientific computing stack makes the library a good candidate to implement the classification and summarization pipeline in the proposed system. Furthermore, its inference API allows the usage of quantized models for CPU inference, making it a low-cost and resource-efficient way to deploy the proposed system.

2.1.3 Retrieval-Augmented Generation

Retrieval augmented generation is a technique for generating knowledge-grounded language using architecture, which was initially formulated and benchmarked by Lewis et al., in their seminal contribution to the Conference on Neural Information Processing Systems of 2020. The core advantage of retrieval augmented generation lies in its ability to solve one of the inherent shortcomings of pure parameterized language generation models, which is their incapacity to access any particular factual data that is not contained within their training parameters. With a retrieval augmentation mechanism integrated into a language generator,

retrieval augmented generation systems are capable of generating output that is rooted in particular documents instead of statistical data learned from pre-training (Jano, 2024).

Under the RAG model, a user-generated query is initially handled by a retrieval component, which converts the query into an embedded vector form and retrieves the most semantically similar document pieces from a vector database where each of these pieces is represented by an embedded vector form. These documents, along with the user query, are used as inputs for generating a response through a generative language model. By doing so, the rate of errors that come with hallucinations in a purely generative model can be reduced, resulting in more accurate and source-driven responses to questions (Han et al., 2024).

Regarding the architecture under consideration, the RAG module represents a different function from the notice summariser pipeline. In this regard, the summarisation algorithm would process notices once they were scraped and generate a summary of them, which would be stored in the database; the RAG module, on the other hand, will allow on-the-go interaction with certain documents that were uploaded by particular users. The significance of this is the fact that the summarisation pipeline operates on notices that have been scraped across all notices available in a database, while the RAG operates on any document (Kiss et al., 2025).

LangChain is a robust, modulated Python library for building RAG pipelines with well-established support for integrating modules for loading documents, splitting texts, generating embeddings, storing vectors in vector stores, and performing inference using language models. The vector store for the solution is ChromaDB which offers efficient approximate nearest neighbour search on the dense embeddings of documents with a compact and file-based backend ideal for early implementation. Finally, Ollama allows inference from open-source large language models like LLaMA 3 and Mistral locally (Farooq et al., 2025).

2.1.4 Web Scraping Technologies

Web scraping refers to the technique of extracting information from websites, using HTTP requests, HTML parsing, and extraction of structured data. As per the system being considered, web scraping seems like the most common technique for data collection, and hence, the design has to be very robust in order to deal with numerous web portals associated with different departments under the government of Nepal. The Python key libraries for scraping in this context include Scrapy and BeautifulSoup4. Selenium and ChromeDriver are complements for the same (Bale et al., 2022).

Scrapy is an ultra-powerful and flexible crawling/scraping framework for the web. It has built-in features for handling concurrent requests, throttling the rate, queuing requests, defining pipelines of data cleaning implemented with middlewares, item validation, and exporting. The particular framework is best suited to large-scale scheduling jobs which require reliability of operation, ethical rate limiting, and data structure. The BeautifulSoup4 package is the instrument to allow parsing HTML more easily and flexibly. Works nicely with Scrapy when it comes to extracting specific information from pages. ChromeDriver and Selenium combine to scrape websites that generate dynamic content using JavaScript (Bhujbal et al., 2023).

2.1.5 Cloud Computing and Deployment Architecture

As per the definition provided by National Institute of Standards and Technology, cloud computing is an on-demand delivery through a network of network-based resources that include applications, servers, networks, storage, and services in a pool, which can be rapidly made available and released. Cloud deployment is the de facto standard in today's web application infrastructure as it provides all the needed attributes, such as scalability, reliability, and geographical reachability, of an information service designed for citizens across the nation (Chang & He, 2025).

The AWS Cloud service is considered to be the most complete cloud offering in terms of services and adoption rate across the world. The suite of services provided by AWS covers the entire infrastructure required for the system architecture presented above. AWS EC2 VM instances will run the NestJS backend API and Python AI microservices, AWS RDS using PostgreSQL as a managed database, and Amazon S3 as object storage for the documents collected and files uploaded by users. The free tier AWS offering suits the purpose of proof of concept development (Kadaskar & Kamthe, 2024).

2.2 Similar Systems and Works

2.2.1 Government Notice Aggregation Platforms

There are several other similar government notice aggregation systems that have been set up elsewhere within other nations and serve as important sources of comparison for the suggested notice aggregation system. The AusTender system set up by the Australian Government aggregates all notices issued by government bodies at the federal level into one database that is searchable and allows for categorisation and email subscriptions for alerts. In the UK, the Government Digital Service uses its publication platform to publish all notices issued in a machine-readable format by the various central government departments.

The effectiveness of such systems illustrates the usefulness and practicability of centralized notice aggregation systems. Nevertheless, their creation was carried out under the condition of quite advanced e-governance systems, with the standardization of data by the government playing a very important role in making the aggregation process much easier. On the other hand, Nepal is characterized by the presence of portals with different designs, diverse formats of documents published on them, and the use of scanned documents (Sharma, 2025).

2.2.2 AI-Powered Document Processing Platforms

Document processing systems like Adobe Acrobat AI Assistant, Microsoft Azure Document Intelligence, and Google Document AI are powerful tools that can be used to extract information from scanned documents through Optical Character Recognition (OCR), information extraction, categorization, and summarization. However, although these products are highly advanced, they are not developed to meet the needs of public information services, and their business model is incompatible with the requirements of the proposed project (Mahadevkar et al., 2024).

A combination of free open-source solutions such as Tesseract OCR, transformers on the Hugging Face platform, and RAG pipelines using LangChain enables a comparable level of functionality without any licensing fees, along with the benefit of customizability in accordance with the unique features of the Nepali government notice portal. The open-source solution stack is integrated into the suggested system for efficiency and future flexibility considerations (Odofoin et al., 2024).

2.2.3 Comparative Analysis and Gap Identification

However, upon conducting a comparative analysis of various techniques that are currently being used for similar purposes, it becomes clear that there is a general trend. The centralised government portals provide the authority and credibility needed, but rely on government competence and desire; commercial AI services provide capabilities, yet their cost prevents them from being accessible to anyone besides paying customers; finally, research prototypes provide innovative solutions but do not reach production level. There is no system available that would combine the three aspects of aggregation, classification and summarization through AI, and answering questions about the documents on a web portal for Nepalese citizens (Albous et al., 2025).

2.3 Technical Research

2.3.1 System Requirements Analysis

Functional and non-functional requirements for the proposed system were formulated through problem context analysis, similar system reviews, and successive refinement of project goals. The system should have four main functions: automatic notices intake, artificial intelligence notices processing, users' access to notices, and Q&A from documents. The mentioned functions impose different demands on the system's technical design. These are discussed in the next sections (Jerab, 2024).

Non-functional requirements will consist of system reliability and availability in line with that expected of an information system accessible by the public, system responsiveness within acceptable limits for searches and retrievals, scalability in relation to increased notice volumes without requiring restructuring of architecture, security and data integrity for uploaded files and subscription data, and adherence to ethical guidelines related to data scraping and protection. All these factors will be used to select the cloud computing strategy and microservice architecture described in Chapter 3.

2.3.2 Hardware Requirements

For the development environment, the minimum requirements for hardware include a processor not less than an Intel Core i5 or better at 2.4 GHz frequency, RAM not less than 8 GB to accommodate browser-based development environment, inference of natural language processing models, and database functions locally, and storage capacity of 256 GB solid state drive for development environment, model files, and samples scrapped data. Internet broadband connectivity is necessary for access to cloud services, data scrapping tasks, and communications through APIs. For the deployment of cloud servers, AWS EC2 instances shall be provisioned with adequate specifications to accommodate computational requirements of the AI inference and API functions (Krasteva et al., 2025).

2.3.3 Software Selection and Justification

The NestJS framework has been chosen for the implementation of the backend API because it has a TypeScript first approach and it makes the code more maintainable and better using the static type checker. The architecture of the NestJS framework resembles that of Angular, making sure that there is a proper segregation of different responsibilities within API routes, services, and the data layer. NestJS supports task scheduling out of the box by means of the `@nestjs/schedule` package that will be needed for periodic task triggers for notice scraping.

Python 3.x will be used in conjunction with NestJS for all the AI and web scraping-related microservices, owing to the fact that the Python environment is highly evolved for all kinds of tasks, including natural language processing model inference, optical character recognition, and web scraping. The integration of NestJS API and the Python microservices will be done using a lightweight RESTful API call or message queue (Biswas et al., 2024).

The frontend framework used is Next.js, which uses React as its base. Next.js comes with an out-of-the-box solution that allows server-side rendering, static site generation, and routes based on files. It was chosen because of its great performance optimization, developer experience due to hot module replacement and TypeScript support, and smooth deployment process with Vercel, which gives the ability to deploy globally with a CDN network without extra configurations.

The PostgreSQL database system running in Amazon RDS is being used as the main relational database system, which is chosen due to its good support for full-text searching using `tsvector` and `tsquery` features that will be useful when searching for notices. PostgreSQL offers good ACID properties, JSON support for storing notices, and good integration with the TypeORM library.

Chapter 3: Methodology

The paper will explicate the strategies and frameworks selected during the research and development of systems. This chapter will discuss the methods, techniques, and approaches that are being adopted in designing and developing an AI-Powered Cloud-Based Public Notice Management System for Nepal. The exposition not only makes clear the development methodology, it justifies the selection of approach and outlines the design for data gathering and analysis strategy applied for defining the system requirements. This chapter is basically meant to do more than simply seeing what each phase's function is; it should be explained what is going on at each phase and how it contributes to fulfilling the project objectives (Mohapatra & Kumar, 2019).

3.1 System Development Methodology

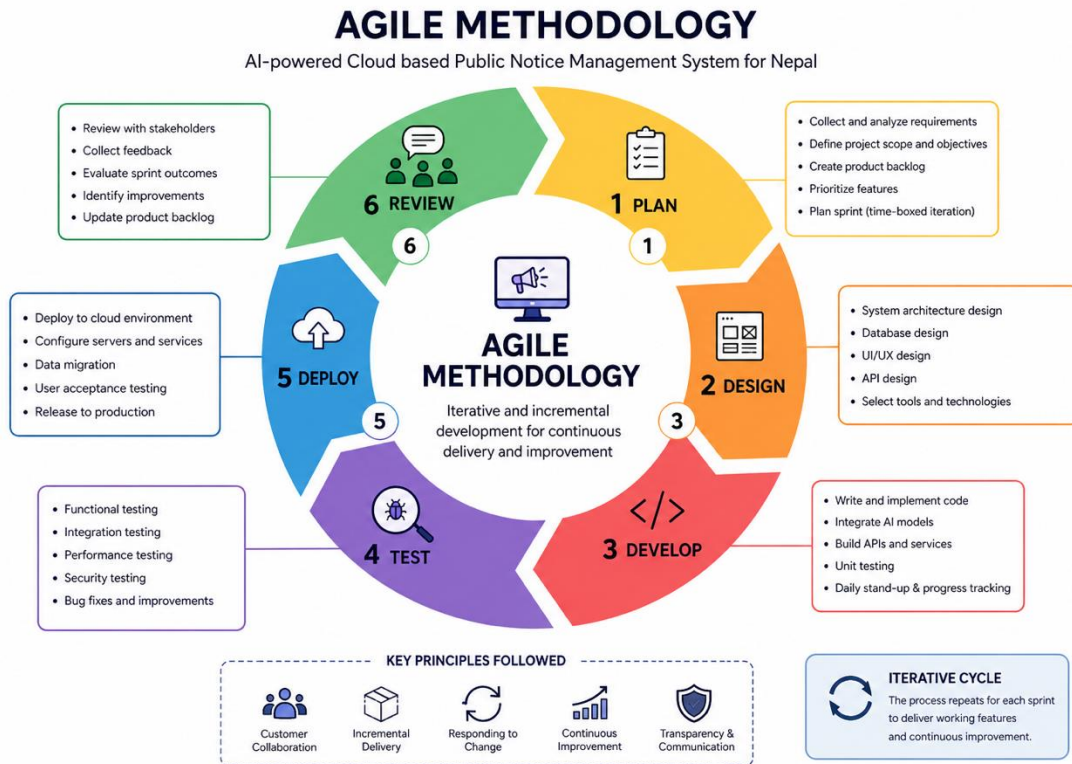
This project has opted for the Agile Software Development methodology. It is a model developed incrementally by iteration. This particular model of software development offers strong emphasis on flexibility, ongoing customer collaboration, and the delivery of working components on a very regular schedule. High contrast to traditional waterfall models, which follow strict progression from requirements to deployment; this particular methodology adapts developments of continuous testing feedback and evolving requirements throughout the life of the project.

The Agile approach was chosen for this project for a number of very good reasons. Firstly, this system comprises several complex and technically diverse components: web-scraping pipelines, OCR processing, NLP-based AI models, a cloud-hosted back-end API, and a responsive front-end application. Any tests run at any level of the development phase for one module may expose requirements or constraints that have implications for another

module. But with Agile, this kind of discovery is facilitated without demanding a complete restart of the development process (Karki et al., 2024).

Figure 1

Agile Methodology



The project talks of those activities that is, usability testing, mockups, HTML coding, and final testing that are highly related to the user-side boxes for search interface, subscription alerts, and document Q&A module, about which it is very essential to validate their usability with iterative feedback, and not a single round of testing on the completion of a project. The Agile model supports a relationship to users operating through continuous feedback and then helps them integrate each of these into every development life cycle in such a way that the final system indeed meets the requirements of the user.

The AI features of this system mostly lie in the domain of classification and summarization pipelines that use Hugging Face models, quintessentially experimental. The accuracy of these models can, therefore, only be estimated based on actual scraped data.

Some adjustments might need to be refined if the performance was observed. Agile's iterative structure paves the way for one to go back to enhance these parts during subsequent phases without causing a setback in the project schedule as a whole (M et al., 2024).

The project is organized into eight consecutive but iterative stages, which are carried out over two semesters. The Investigation Report is the deliverable of Semester 1 and encompasses Phases 1 to 4, whereas Semester 2 shall be dedicated to the execution of the project.

3.1.1 Development Phases

This is followed by a brief explanation of the eight stages of development and the tasks involved at each stage.

Phase 1: Project Planning and Environment Setup

This is next is finalizing the architecture of the system, definition of the functional and non-functional requirements including setting up the local development environment. It involves installing and configuring Node.js with NestJS for the backend API and Scrapy, Selenium, Hugging Face Transformers, LangChain, and ChromaDB for the Python environment, before finally scaffolding the Next.js frontend project. This phase will give a start to provisioning the cloud environment in AWS, which would include the setting up of EC2 instance configurations and RDS (PostgreSQL) and S3 buckets. Version control will establish core project repository structure (Cuskley et al., 2024).

Phase 2: Web Scraping Module Development

The phase encompasses the development of scraping spiders in Scrapy for the government portals serving static HTML content. Selenium with ChromeDriver emulates

browser behavior to download the rendered HTML for those serving content dynamically by JavaScript. The OCR pipeline uses Tesseract to extract text from scanned PDF notices and image-embedded documents. It will be implemented with rate-limiting, robots.txt compliance, and user-agent configuration systems for ethical scraping. Noticing is done, normalized, and stored in PostgreSQL through NestJS backbone.

Phase 3: Cloud Data Pipeline

After which, a cloud-based data pipeline was set up which would facilitate storage, management, and retrieval of all the scraped notices. The primary database for the project was 'RDS PostgreSQL.' This enabled the process of notice retrieval, which is achieved using PostgreSQL's native full-text search, tsvector, and tsquery. Original notice documents in PDF and image format would be stored in an Amazon S3 Bucket. The backend, built in NestJS, provided REST endpoints for fetching the frontend and also took care of search, filter, and alert operations. The scheduled periodic scraping jobs are just scheduled tasks using the help of the @nestjs/schedule package.

Phase 4: AI Classification and Summarisation Integration

This involves the addition of an automatic notice classifier and abstractive summarizer to the Python-based microservice, which were used with pre-trained Hugging Face Transformer models. As a result, every notice fetched by the scraping pipeline will be run through the classification model in order to assign it to one of the five predefined categories: Jobs, Exams, Tenders, Policy, or Other. A different model for summarization, this time based on either BART or T5 architecture, is responsible for creating an intelligible, plain-language summary of the contents of that notice. The Python microservice communicates to the outside world via RESTful calls to the NestJS API. The model's performance will be checked on a test set for the scraped notices and refined further (Vyshotravka et al., 2025).

Phase 5: Frontend Development

The web interface that the citizens face was developed using Next.js, and server-side rendering was done for performance optimization. The frontend consists of a centralized search and filter interface, an individual notice page that shows the AI-generated summary along with the original source link, user account registration and login, and subscription management for alerts based on keywords and categories. It is responsive, so there would be no issue accessing from a desktop or a mobile browser (Ekpobimi, 2024).

Phase 6: Retrieval-Augmented Generation Module

The RAG gearing has been engineered using LangChain for pipeline orchestration and ChromaDB as its vector store for document embeddings. The user's method of uploading a PDF or image document is from which it is to be preprocessed, chunked, embedded and stored in ChromaDB. The input to this module is a natural language question from the user. The user submits a natural language query, its embedding is taken and, hence, with this, the most semantically relevant document chunks are retrieved. These chunks are considered in association with the query to a generative language model (LLaMA 3 through Ollama), which provides a response anchored in sources. The implementation module acts independently from the notice summarization pipeline and is actually designed for on-demand interaction with documents (Patel, 2023).

Phase 7: Testing and Cloud Deployment

This phase will be about checking the end-to-end working of the system. If this works fine, we go to individual function testing. First, integration will be tested; if that goes fine, further processing would be done in user acceptance Testing. In it, proper handling of real/relevant data with high processing speed in backend and frontend activity along with the responsiveness of the system will be tested. If everything runs well, the back end service will

be deployed to AWS EC2 and the front end to Vercel for global CDN distribution. A testing dataset of 200-300 notices across all five categories is imported into the production database for evaluation in the production environment.

Phase 8: Documentation and Finalization

In the last stage, the task involves the production of project documentation which includes the Final Year Project report, API documentation, deployment documentation, and user guides. The Gantt chart is revised to show the actual work done against the planned one. All appendices, logs, and ethics documents are included with the report.

3.1.2 System Architecture

The AI-Powered Cloud-Based Public Notice Management System shows modular cloud-based system architecture for easy scalability, maintainability, and efficient processing of public notices collected from multiple Nepalese government portals.

The process gets initiated through data sources, where notices are collected from government sites in the form of HTML pages, PDF files, and image-based documents. Web scraping technologies used in this data collection layer are Scrapy, BeautifulSoup4, and Selenium for extracting information from websites that are both static and JavaScript rendered (Witanto & Oktian, 2022).

The processing layer handles OCR by leveraging Tesseract on scanned PDFs and images to obtain text with subsequent text-cleaning and normalization. This is then followed by notice classification, AI summarization, category tagging, and duplicate detection in the AI intelligence layer, using transformer-based NLP models from the Hugging Face ecosystem.

The output data is stored in PostgreSQL by a Supabase database; user info, notifications, categories, sources, and uploaded documents.

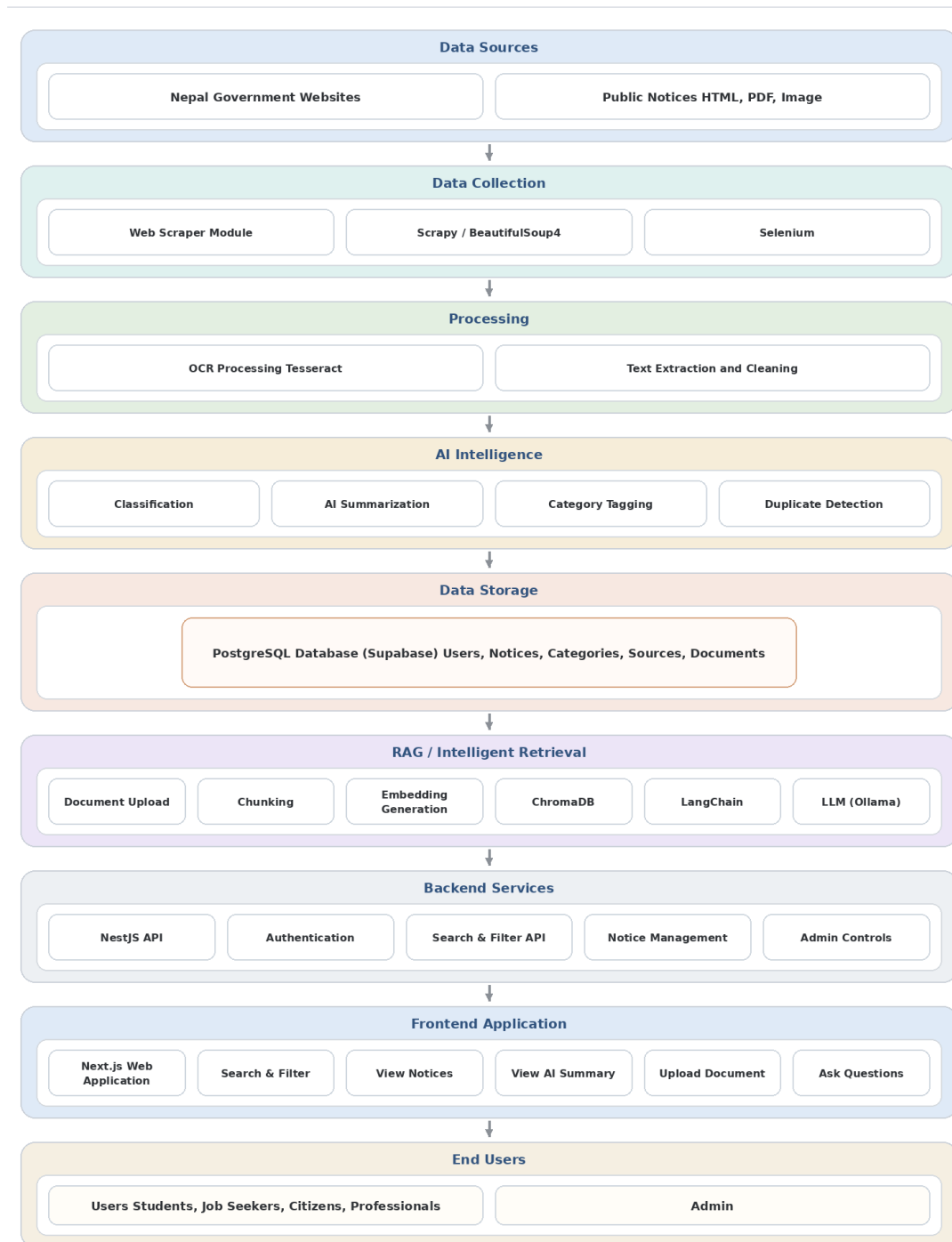
When a document is uploaded to Intelligent Document Processing, it gets chunked, then it has its embeddings created by the RAG module with ChromaDB, a combination of LangChain, and LLMs based on Ollama for the Q&A module (Kanaan et al., 2019). Backend development utilizes NestJS for performing tasks such as user authentication, searches and filters, managing notices, and administration. Meanwhile, the front-end application uses Next.js to offer capabilities like searching notices, reading AI summaries, uploading documents, and smart questioning.

Lastly, the overall solution will be used by end-users including students, job seekers, citizens, professionals, and even administrators.

Figure 2

System architecture diagram

System Architecture Diagram



Scrapy

Scrapy is the most popular web scraping framework developed to collect public notices from websites of the Government of Nepal. It assists in automatically sending requests to websites and extracting structured information; probably, the title of a notice, its

publishing date, source link, and notice content will be stored in a well-defined order. This is quite helpful in large scale scraping tasks due to support for scheduling, rate limiting, crawling through multiple pages, and ethical scraping practices.

BeautifulSoup4

It is very helpful in parsing HTML markups and extracting elements from webpages. When deep HTML parsing of a webpage is required, BeautifulSoup4 can be combined with Scrapy. For example, if nesting goes deep inside tags for notice titles or tables and metadata, BeautifulSoup will help in extracting information correctly; this makes extraction of data neat and flexible.

Selenium

Selenium comes in handy in a situation whereby the content of the government websites loads through JavaScript dynamically. Contrary to Scrapy, which mainly focuses on static pages, Selenium is a browser emulator that will wait until the content fully loads before performing extraction. This becomes very relevant when the modern government portals are constructed in such a way that the notices appear only after page rendering or button interaction.

Tesseract OCR

It will be useful for the OCR processes. A good number of government notices in Nepal come in the form of scanned PDFs or images besides text documents. So, Tesseract plays a major role in converting such scanned files into readable texts to allow the system to handle, classify, summarize, and search the content of notices effectively.

Transformers of Hugging Face

Hugging Face provides pre-trained models in NLP for AI text classification and summarization. It offers a service for notifying automatically whether a notice is related to Jobs, Exams, Tenders, Policy, or Other and gives a short, plain-language summary about that notice to enable a user to quickly know what that long and complex government notice is about, without the need to read the whole thing.

PostgreSQL (Supabase)

PostgreSQL is the master relational database that houses notices, user data, categories, uploaded files, and subscriptions. Supabase is a cloud-based tool for managing PostgreSQL with no headaches or hassle of ensuring data storage and very fast access for search, filter, and structuring management of all system data.

ChromaDB

The vector database in which the RAG module is run is known as ChromaDB, in which an uploaded document gets sliced into chunks to produce embeddings. This will allow the retrieval of semantically relevant parts of the document upon user query, hence speeding up the process of question answering from documents.

LangChain

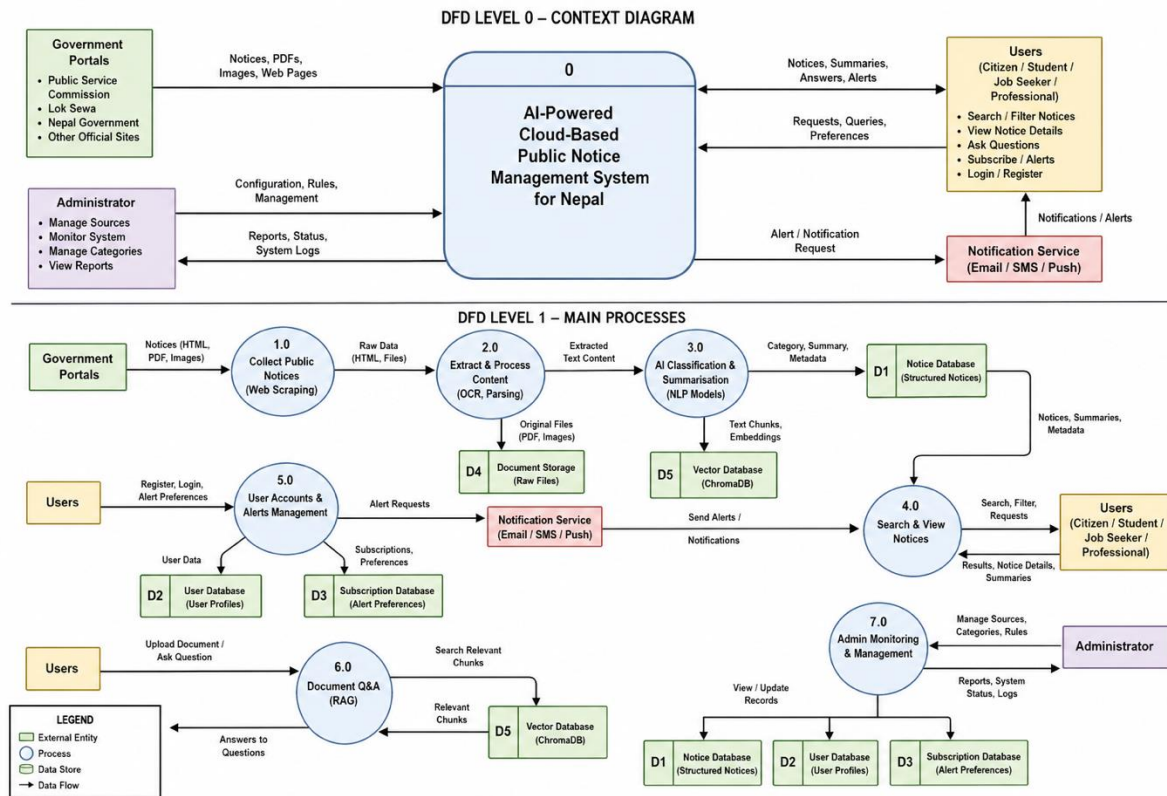
This is where LangChain comes in: it is supposed to allow users to apply the RAG workflow without a glitch in the document loader, text chunker, embeddings, vector searcher, language model response generator chained into one pipeline. Before routing the user question to the correct context of the document, it assures that the document is necessary for any final answer generation.

3.1.3 Data Flow Diagram

DFD illustrates the process of general working in the AI-Powered Cloud-Based Public Notice Management System for Nepal. Level 0 demonstrates how the primary system interacts with external entities, including government portals, users, administrators, and notification services. The government portals provide public notices in HTML pages, PDFs, or images; users search notices, summaries, and alerts, and document-based answers to questions will be returned. The system splits into significant processes at Level 1, including notice collection through web scraping, OCR-based content extraction, AI classification and summarization with NLP models, notice search and view, user account and alert management, document-based question-answering with RAG, and monitoring by administrators. PostgreSQL and ChromaDB are some of the central databases where processed data is stored for faster retrieval and safe storage with intelligent access to government notices.

Figure 3

Data flow diagram



3.2 Data Gathering Design

The data collection for this project is a mix of both, using that of primary origin done through feedback from users and semi-structured interviews and the second being systematic web scraping of government portals. Every technique has its own way of making a contribution to processes pertaining to inform design of the system and giving support for requirements validation. All primary data collection activities have the approval to start by project supervisors, having submitted relevant documents for ethical clearance before beginning the data collection exercise (Khder, 2021).

3.2.1 Survey Design

I developed an online survey that aimed to collect the quantitative data on the current experience of a large sample of potential end users in accessing government notices and their

expectations from a single platform. That is, at least 30 participants in each of the four identified major target groups outlined in Chapter 1: students, job seekers, working professionals, and contractors in Nepal who frequently engage with government notifications (Torrentira & Torrentira, 2020).

The survey was disseminated digitally through university networks and community groups. The purpose was to reach diverse respondents with different geographic locations within Nepal, both urban and semi-urban locations. Appropriate briefings were done for all participants on the voluntary and anonymous nature of the survey participation following the ethical clearance obtained. Demographics of the respondent, which include their gender, age group, occupation, were compiled and submitted in the appendix as per guideline. Table 3.2 depicts the survey questions.

3.2.2 Interview Design

A few semi-structured interviews were carried out with a minimum of three respondents to permit a more parsimonious in-depth interpretation of user behavior, pain points, and expectations that would complement the findings of the quantitative survey. On the system-relevant side, the students looking for exam and scholarship notices quite often, a working professional/contractor monitoring tender and procurement notices, and a public administration or IT professional with knowledge about the e-governance landscape of Nepal.

Semi-structured interviews were found to be the most suitable technique for letting the conversation flow into unexpected themes and experiences uncommonly covered by fixed questionnaires; in principle, over completely structured interviews. Each interview was conducted with the knowledge and acceptance of the participant, who was assured that their identity would be kept totally confidential (Karatsareas, 2022).

3.2.3 Secondary Data Collection via Web Scraping

The secondary data collection for this research study is done through web scraping of certain websites of Nepalese Government organizations. Web scraping accomplishes two tasks: it helps to provide real world notice data to check whether scraping works technically or not and it is used to fill up the test database required for testing of AI classification and summarization. Websites scraped include the following:

- Nepal Public Service Commission (psc.gov.np) for civil services and examination announcements
- Nepal Ministry of Finance (mof.gov.np) for tenders, procurement, and financial policies
- Nepal Judicial Service Commission (jsc.gov.np) for judicial recruitment and administrative announcements
- Tribhuvan University, Controller of Examinations (tucoe.edu.np) for examination dates and results announcements
- Prime Minister and Council of Ministers (opmcm.gov.np) for policy guidelines and administrative announcements

While making my best efforts, I scrape static HTML websites using Scrapy wherever possible, whereas in the case of dynamically JavaScript rendered sites, I use Selenium along with Chrome Driver. Passing the text of PDF notices through the machine-readable process to make them accessible involves laying them out in the Tesseract OCR pipeline. I check the robots.txt file in the root domain of the site targeted and introduce rate limits to avoid overloading the server. In all cases, I have approached it with an explicit user-agent, which means there should be an absolute minimum of 200-300 notices that could be gathered for test data across the entire five categories of notice.

3.3 Analysis

Data analysis is fundamentally a data-driven process that first identifies accurate user needs through quantification and qualification techniques on survey data, interviews, or scrapings. Further triangulation of the findings from the three sources should point towards some common themes or assumptions waiting for validation to prove that the system indeed designed to fulfill real needs of the end-users (Dibekulu, 2020).

3.3.1 Quantitative Analysis Approach

The analysis of survey responses utilizes the application of descriptive statistics. Frequency distributions and percentages are computed for multiple-choice questions in order to establish the most common behavior and preference of users. The Likert-scale answers were summed for each question in the survey to get an average score of satisfaction and also to establish the importance given to certain specific system features by users. The quantitative key findings of this study are: Frequency of visits to the Government portal, consequently providing information on challenges faced most frequently in accessing it; perceived usefulness of AI-generated summaries; and the desired notification method for new notices. This feeds directly into functional requirement prioritization for this system.

3.3.2 Qualitative Analysis Approach

The process started with an analysis of interview transcripts via a thematic approach. The key recurring themes from the participants were identified, coded for each of the variables, and then arranged systematically under thematic categories.

Several major themes came out of the interviews: time shortage that led to visiting several government portals to check notifications; non-availability of notifications from scanned PDFs; low digital literacy by rural-based users, who might need plain-language summaries. Other issues that were raised included concerns over the accuracy of content and

the trustworthiness of AI-generated content. These qualitative insights were instrumental in furthering the validation of quantitative findings, hence adding depth and context to the user requirements identified during the survey (Zairul, 2025).

3.3.3 Preliminary User Requirements

From the analysis of the survey, interview findings, and other technical insights in secondary data collection, the preliminary user requirements have been identified and summarized. These requirements will further be classified into functional requirements (FR) and non-functional requirements (NFR), and will be treated as the baseline specification during the design and implementation of the system.

Table 3.4:

Preliminary User Requirements for the AI-Powered Public Notice Management System

Req. ID	Requirement Description	Priority
FR-01	The system should schedule automatic scraping of public notices from no less than five government portals.	High
FR-02	In the scanned PDF notices, a system should extract text using OCR, more precisely Tesseract.	High
FR-03	Categorize into predefined categories: jobs, exams, tenders, policy, or other, for each scraped notice.	High
FR-04	For each such notice, the system offers a brief AI-powered abstractive summary with the help of models from Hugging Face.	High

Req. ID	Requirement Description	Priority
FR-05	Users will be able to find notices based on keywords, category, date range, and source organization.	High
FR-06	Users can subscribe to keyword-based or category-based alerts for published notices.	Medium
FR-07	Users can upload a PDF or image document and get answers to their natural language questions through the RAG module.	Medium
FR-08	The system will provide a unified responsive web interface to be accessible from desktop and mobile browsers.	High
FR-09	Support user account registration and login feature for personalized features.	Medium
NFR-01	Availability will be maintained at a minimum of 99% during scraping and notice retrieval operations.	High
NFR-02	Search query responses will be returned within three seconds under normal load conditions.	Medium
NFR-03	The system shall comply with the robots.txt guidelines of all the government portals that are scraped.	High

The requirements listed in Table 3.4 correspond to the explicit needs conveyed by participants in the survey and interviews and to the technical constraints obtained from the literature review and feasibility assessment of web scraping. High-priority requirements will

be implemented in the core phases of implementation Phases 2 through 5 and medium-priority requirements will be integrated into the next phases, provided the project progresses. They will be further refined and validated during the UAT phase of software development in an AGILE cycle.

Chapter 4: Conclusion

4.1 Achievement of Investigation Phase Objectives

The research findings at this juncture have sufficiently met the requirements of the first phase of the Final Year Project in establishing a well-evidenced research foundation on designing and developing an AI-Powered Cloud-Based Public Notice Management System for Nepal. The investigation phase has clearly delivered core deliverables: an extensive, multi-faceted literature survey; a clear, unambiguous problem statement; definition of project objectives that are set out clearly and operationally; an elaborate system design, backed by eight architectural and design diagrams; justification for the technical stack in full; and designing primary data-collection instruments for the user research activities that are upcoming (Limiany et al., 2024).

Chapter 2's literature review argued that the issue at hand has significant civic interest and that its lines of solution have a degree of technical maturity. The e-governance literature review, together with the researches on public information systems, provided a sound theoretical framework for understanding the structural causes of the fragmentation problem within the government information ecosystem in Nepal and documented ways through which AI-enhanced information services could generate measurable public value. The summary and review of NLP technologies for text classification and summarization show that the Transformer models, as available through the Hugging Face ecosystem, are technically powerful enough to do this given the appropriate model selection and evaluation procedures in place.

The review of retrieval-augmented generation technology confirmed that the LangChain, a ChromaDB-based RAG architecture, was used for the document interaction

module. It was technically sound and well-documented, suitable for this proof-of-concept use case.

Further, confirmation of the originality was found in the comparison of related systems, under 2.2, within the specific context of Nepali government notice aggregation. No such pre-existing platform was traced that embodies multi-sourced aggregation, AI classification and summarization, and RAG-based document interaction on the citizen-facing interface for the public information context in Nepali (Dimitrova, 2025).

The methodology chapter lays down a clearly outlined Agile development plan consisting of eight distinct but iteratively related phases. It has a nicely designed primary data collection strategy that combines a validated 30-participant survey with three semi-structured interviews, and a rigorous analysis framework that draws heavily on descriptive statistical methods and systematic thematic analysis. All the above would form a coherent and comprehensive blueprint for work on implementation to come, through the preliminary system design produced during this phase, consisting of the system architecture, Level 0 and Level 1 data flow diagrams, use case diagram, component diagram, sequence diagram, deployment diagram, and entity relationship diagram.

4.2 Research Gaps and Areas for Further Development

There have been detected several research gaps and areas that would require another round of investigation during the implementation phase. Much more important is that the performance of transformer-based NLP models developed for Nepali government notice text has never been empirically tested. Most publicly available transformer models have been pre-trained on general-purpose and news-wire text distributions. However, government notices from a Nepali administrative body are written in a highly formal, specialized bureaucratic

register that could be quite different. A labeled corpus for Nepali government notices might need to be domain-specific fine-tuned in order to realize classification accuracy and summarization quality meeting the standards that would be expected from any public-facing information service (Shahi & Sitaula, 2022).

Moreover, the limitation of scope in the system at its current state applies to the coverage of notices published in the Nepali language or in a mixed Devanagari-English format. Many government notices in Nepal are published entirely or partly in the Nepali script, but the scope for the current project has been explicitly limited to processing AI for English and transliterated text. A fully representative system, level playing field for every citizen, would need multilingual NLP capabilities extending up to the processing of Devanagari script, which has separate challenges to tokenization, morphological analysis, and availability of pre-trained models for Nepali as a low-resource language. This represents a priority direction for future research and development beyond the current project.

There is a lack of clarity around the legal and regulatory frameworks on automatic data access to the websites of the Nepal government. With appropriate legal advice on data governance, copyright laws, and e-government policy in Nepal, it could be said with some safety that the system proposed would maintain ethical scraping practices according to global standards, including compliance with robots.txt, very conservative rate limits, and full transparency in its identification. There isn't a clear set of provisions in Nepalese law at this point on automatic retrieval and republication of government information (Giri S. , 2020).

Fourthly, a project has within its scope the setting up of an academic project while the long sustainability of the system remains out of scope. The dynamism of governmental website structures also involves changes every now and then, requiring a change in the scraping configurations. AI models will have to be retrained regularly or replaced if better alternatives are discovered. But, operational entailments for continued prudent cost

management, monitoring and maintenance of security for cloud infrastructure remain. It is thus completely dependent on the responsible maintenance entity as to how useful the system will be in the long run. The sensitive fact of overnight changes would demand both technical capacity and institutional commitment. Thus, on the far end of operational entailments, the long-term utility of the system essentially relies on a responsible maintenance entity, for which a combination of technical capacity and institutional commitment is required. Another has been the identification and engagement of potential institutional partners for future system stewardship, an important consideration for the project's post-delivery impact.

4.3 Reflection on the Investigation Phase

The investigation phase was really good and rewarding from an intellectual point of view since it deepened my insight into both the problem domain and the technical solution space. Research conducted documented that the dimensions of the landscape of e-governance and NLP were not apparent at the beginning of the project. Most specifically, it was demonstrated that the problem of civic access in Nepal is a subset of structural shortcomings in e-governance, which have been well documented in international literature and overcome in other national contexts by the kind of dual approach postulated for this project: integrated aggregation and AI enhancement.

The most worthwhile exercise in preparing the system design diagrams has been in exposing integration challenges and design decisions not apparent at the high-level conceptual stage. For example, it was practically impossible to detail the sequence diagram, and it took careful analysis to appreciate that the workflow was asynchronous between the scraping trigger in OCR processing, AI inference, and database write operations. That led to adopting a task queue architecture at that point since it had not been explicitly specified in the preliminary system design. Such design refinements capture the iterative nature of the

investigation process and the need for detailed diagrammatic analysis as a complement to high-level architectural design (Al-Fedaghi, 2021).

There is a high level of anticipation for the implementation phase; consequently, the key data collection activities planned for early Phase 1 will most likely be critical grist for the mill in defining the ultimate usability and relevance of this system to citizens. The survey and interview results will underpin the empirically sound interface design decisions that will then go on to mold the way in which citizens experience and interact with the platform. It is expected that all these activities will validate the core feature set that was identified during the investigation phase as being part of a specific design subtlety and priority adjustments to be included in the development plan. The Investigation Report provides strong evidence-based and technically stringent underpinnings for this next phase of work (Tarhini et al., 2015).

References

- Nchaga, A. M. (2025). Civic Engagement in the Digital Age: Challenges and Opportunities. *Public Administration Kampala International University*, 6(1), 58-65.
<https://doi.org/10.59298/NIJCIAM/2025/6.1.5865>
- Shrestha, D., Devkota, B., & Jeong, S. R. (2023). Challenges and Factors affecting E-governance practices in Nepal. *9th International Conference on Software, Knowledge, Information Management and Applications*, 9(1), 15-17.
<https://doi.org/10.1109/SKIMA.2015.7399981>
- phuyal, P. (2024). Digitalization of Government Services and Citizen. *prashasanjournal*, 135-152(1), 139. <https://doi.org/10.3126/prashasan.v56i1.67336>
- Nirman, P. (2025). Barriers to digital participation in developing countries: Identifying technological, social, and cultural obstacles to community involvement. *GSC Advanced Research and Reviews*, 23(2), 061-071.
<https://doi.org/10.30574/gscarr.2025.23.2.0130>
- Rayamajhi, S. (2026). Digital Public Infrastructure in Nepal: Progress, Problems, and the Path to 2030. *Rayznews*, 2(1), 120-130. <https://doi.org/10.13140/RG.2.2.35892.31366>
- Elen, A. (2023). A Complete Solution to Exam Scheduling Problem: A Case Study. *Journal of Scientific Reports*, 49(2), 12-34. <https://doi.org/10.15130/RG.2.4.35592.31346>
- Miah, M. (2024). Digital Inequality: The Digital Divide and Educational Outcomes. 2(1), 15-25. <https://doi.org/10.27140/RG.2.2.24892.21345>
- Jagadish, H. V., Gehrke, J., Labrinidis, A., & Papakonstantinou, Y. (2023). Big Data and Its Technical Challenges. *Communications of the ACM*, 7(1), 86-94.
<https://doi.org/10.1145/2611567>

- Bagale, S. (2025). Digital Administration: Strategies for Transforming. *A Peer-Reviewed, Multidisciplinary and Multilingual Journal*, 8(2), 34-39.
<https://doi.org/https://doi.org/10.3126/mg.v8i1.84191>
- Wrange, p., Ebbesson, J., & Hey, E. (2023). 16 - SDG 16: Promote Peaceful and Inclusive Societies for Sustainable Development, Provide Access to Justice for All and Build Effective, Accountable and Inclusive Institutions at All Levels. 5(1), 23-30.
<https://doi.org/https://doi.org/10.2156/mg.v8i1.83138>
- Bista, I. (2023). Issues with Development Projects in Nepal: A Case Study of the Development Projects in Nepal and its Issues Using the Agent Based Approach (ABA). *Journal of Systems Thinking*, 2(1), 20-28. <https://doi.org/10.54120/jost.2025-0066IB>
- Shakya, S., Basnet, A., Sharma, S., & Gurung, A. (2023). Optical Character Recognition for Nepali, English Character and Simple Sketch Using Neural Network. *Recent Advances in Information and Communication Technology*, 6(1), 45-53.
https://doi.org/10.1007/978-3-319-40415-8_6
- Al-Zoubi, Q. (2009). Mobile-Based Notification System for University's Events. *University Utara Malaysia*, 3(1), 20-57. https://doi.org/10.1007/958-3-329-20415-8_3
- Giri, P. B., Giri, U. R., Giri, B. R., & Giri, A. (2025). E-GOVERNANCE AND SERVICE DELIVERY IN NEPAL: EVIDENCE FROM NAGARJUN MUNICIPALITY. *International Journal of Managing Public Sector Information and Communication Technologies*, 2(1), 1-14. <https://doi.org/10.5121/ijmpict.2025.16401>
- Veturi, S., Vaichal, S., Tripto, N. I., & Jagadheesh, R. L. (2024). RAG based Question-Answering for Contextual Response Prediction System. *The Pennsylvania State University*, 6(1), 126-132. <https://doi.org/10.48550/arXiv.2409.03708>

- Efthymiou, I.-P. (2025). The Role of E-Government and E-Governance in Modern Societies. *Public Governance Practices in the Age of AI*, 7(1), 45-68.
<https://doi.org/10.4018/979-8-3693-9286-7.ch003>
- Janssen, M., Charalabidis, Y., & Zuiderwijk, A. (2012). Benefits, Adoption Barriers and Myths of Open Data and Open Government. *Information Systems Management*, 29(4), 258–268. <https://doi.org/10.1080/10580530.2012.716740>
- Wirtz, B. W., Weyerer, J. C., & Gever, C. (2018). Artificial Intelligence and the Public Sector—Applications and Challenges. *International Journal of Public Administration*, 42(7), 67-79. <https://doi.org/10.1080/01900692.2018.1498103>
- Kader, S. A., Noor, F., & Akter, F. (2025). E-Governance in the Modern World: A Global Perspective. *Department of B. S Political Science and Public Administration*, 6(2), 16-30.
- Omweri, F. S. (2025). Digital Inclusion and Institutional Effectiveness in E-Governance for SDG Delivery in the Global South: A Scoping Review. *INTERNATIONAL JOURNAL OF RESEARCH AND INNOVATION IN SOCIAL SCIENCE*, 9(9), 2454-6186.
<https://doi.org/https://dx.doi.org/10.47772/IJRISS.2025.909000251>
- Ahmed, A., Aswad, F., Alsaeedi, M. K., & Khudhair, I. (2025). An Improved Hybrid GRU and CNN Models for News Text Classification. *INTERNATIONAL JOURNAL ON INFORMATICS VISUALIZATION*, 9(1), 69-79. <https://doi.org/http://dx.doi.org/10.62527/joiv.9.1.2658>
- Varade, S., Sayyed, E., Nagtode, V., & Shinde, S. (2021). Text Summarization using Extractive and Abstractive Methods. *ITM Web of Conferences*, 5(3), 78-89.
<https://doi.org/10.1051/itmconf/20214003023>

- Jano, P. W. (2024). Retrieval-Augmented Generation Systems: A Comprehensive Survey of Architectures, Applications, and Future Directions. 9(8), 34-45.
<https://doi.org/10.1051/itmconf/20214003023>
- Han, H., Wang, Y., Shomer, H., Guo, K., Ding, J., Lei, Y., Mahantesh Halappanavar, R. A., Long, B., Zhao, T., Neil Shah, A. J., Xia, Y., & Tang, J. (2024). Retrieval-Augmented Generation with Graphs (GraphRAG). *Information Retrieval (cs.IR); Computation and Language*, 10(4), 25-45.
<https://doi.org/https://doi.org/10.48550/arXiv.2501.00309>
- Kiss, C., Nagy, M., & Szilágyi, P. (2025). Max–Min semantic chunking of documents for RAG application. *Discover Computing*, 28(1), 46-48. <https://doi.org/10.1007/s10791-025-09638-7>
- Farooq, A., Anuragi, D., Li, Z., & Ziat, M. (2025). Securing local LLMs for academic research: a human-system integration analysis and evolution of TAUCHI-GPT. *Human-Intelligent Systems Integration*, 7(1), 53-85. <https://doi.org/10.1007/s42454-025-00085-9>
- Bale, A. S., Ghorpade, N., Kamalesh, S., & S, R. (2022). Web Scraping Approaches and their Performance on Modern Websites. *Third International Conference on Electronics and Sustainable Communication Systems*, 8(1), 25-56.
<https://doi.org/10.1109/ICESC54411.2022.9885689>
- Bhujbal, M. V., Bibawanekar, M. B., & Deshmukh, P. (2023). News Aggregation using Web Scraping News Portals. *International Journal of Advanced Research in Science Communication and Technology*, 3(2), 2581-9429.
<https://doi.org/10.48175/IJAR SCT-12138>
- Chang, C.-Y., & He, X. (2025). The Liabilities of Robots. *Text*, 14(8), 56-76.
<https://doi.org/10.2139/ssrn.5159436>

Kadaskar, H. R., & Kamthe, V. R. (2024). An overview of AWS. *International Journal of Scientific Research in Modern Science and Technology*, 3(7), 22-30.

<https://doi.org/10.59828/ijrmst.v3i7.223>

Sharma, B. P. (2025). Use of Information and Communication Technology (ICT) and E-Governance in Dhangadhi Sub-Metropolitan City, Nepal. *NPRC Journal of Multidisciplinary Research*, 2(10), 40-63.

<https://doi.org/10.3126/nprcjmr.v2i10.85865>

Mahadevkar, S. V., Patil, h., Kotecha, K., & Soong, L. W. (2024). Exploring AI-driven approaches for unstructured document analysis and future horizons. *Journal of Big Data*, 11(1), 36-54. <https://doi.org/10.1186/s40537-024-00948-z>

Odofin, .., Abayomi, A. A., Ogbuefi, E., & Ogeawuchi, J. C. (2024). Strategic Integration of LangChain, Hugging Face Transformers, and OpenAI for Document Intelligence Systems. *International Journal of Scientific Research in Science Engineering and Technology*, 11(4), 413-426. <https://doi.org/10.32628/IJSRSET25121177>

Albous, M. R., Al-Jayyousi, O. R., & Stephens, M. (2025). AI Governance in the GCC States: A Comparative Analysis of National AI Strategies. *Journal of Artificial Intelligence Research*, 82(15), 2389-2422. <https://doi.org/10.1613/jair.1.17619>

Jerab, D. A. (2024). The Impact of Digital Transformation on Public Services. *Journal of Artificial Intelligence*, 9(5), 86-94. <https://doi.org/10.1613/jair.1.17619>

Krasteva, V., Stoyanov, T., & Jekova, I. (2025). Implementing Deep Neural Networks on ARM-Based Microcontrollers: Application for Ventricular Fibrillation Detection. *Applied Sciences*, 15(4), 19-65. <https://doi.org/10.3390/app15041965>

Biswas, R., Jana, S., & Pal, K. (2024). Leveraging Python for Artificial Intelligence: A Comprehensive Exploration. *SIRJANÄ —A Journal on Arts and Art Education*, 54(2), 178-185.

- Mohapatra, S., & Kumar, A. (2019). Developing a Framework for Adopting Artificial Intelligence. *International Journal of Computer Theory and Engineering*, 11(2), 19-22. <https://doi.org/10.7763/IJCTE.2019.V11.1234>
- Karki, S., Bhoj, R., & Karki, S. (2024). Artificial Intelligence for Digital Marketing Transformation in Nepal: A Review. 5(1), 25-86. <https://doi.org/10.7763/IJCTE.2019.V11.1234>
- M, A., Danda, a., & Singh, R. P. (2024). Automation of Text Summarization Using Hugging Face NLP. *5th International Conference for Emerging Technology (INCET)*, 14(5), 45-58. <https://doi.org/10.1109/INCET61516.2024.10593316>
- Cuskley, C., Woods, R., & Flaherty, M. (2024). The Limitations of Large Language Models for Understanding Human Language and Cognition. *OpenMind*, 9(1), 1058-1083. https://doi.org/10.1162/opmi_a_00160
- Vyshotravka, D., Vysotska, V., HU, Z., & Uhryn, D. (2025). Smart Application for Recruiting Based on Natural Language Processing Methods, Transformer Models and Siamese Neural Network Architecture. *International Journal of Intelligent Systems and Applications*, 17(5), 95-157. <https://doi.org/10.5815/ijisa.2025.05.07>
- Ekpobimi, H. (2024). Building high-performance web applications with NextJS. *Computer Science & IT Research Journal*, 5(8), 1963-1977. <https://doi.org/10.51594/csitrj.v5i8.1459>
- Patel, V. (2023). Analyzing the Impact of NextJS on Site Performance and SEO. *International Journal of Computer Applications Technology and Research*, 12(10), 24-27. <https://doi.org/10.7753/IJCATR1210.1004>
- Witanto, E. N., & Oktian, Y. E. (2022). Toward Data Integrity Architecture for Cloud-Based AI Systems. *Symmetry*, 14(2), 273. <https://doi.org/10.3390/sym14020273>

- Kanaan, R., Abumatar, G., & Mohammed, A. (2019). Cloud-Based Management Information System: A Systematic Review and Future Research Scope. *Journal of Social Sciences*, 8(3), 509-525. <https://doi.org/10.25255/jss.2019.8.3.509.525>
- Khder, M. (2021). Web Scraping or Web Crawling: State of Art, Techniques, Approaches and Application. *International Journal of Advances in Soft Computing and its Applications*, 13(3), 145-168. <https://doi.org/10.15849/IJASCA.211128.11>
- Torrentira, M. C., & Torrentira, M. J. (2020). ONLINE DATA COLLECTION AS ADAPTATION IN CONDUCTING QUANTITATIVE AND QUALITATIVE RESEARCH DURING THE COVID-19 PANDEMIC. *European Journal of Education Studies*, 7(11), 78-87. <https://doi.org/10.46827/ejes.v7i11.3336>
- Karatsareas, P. (2022). Semi-Structured Interviews. *Research Methods in Language Attitudes*, 8(1), 99-113. <https://doi.org/10.1017/9781108867788.010>
- Dibekulu, D. (2020). An Overview of Data Analysis and Interpretations in Research. 8(2), 52-61. <https://doi.org/10.14662/IJARER2020.015>
- Zairul, M. (2025). Mastering Thematic Analysis: A Step-by-step Guide for Beginners With Tips for Systematic Analysis Using ATLAS.ti 25. *International Journal of Qualitative Methods*, 24(7), 25. <https://doi.org/10.1177/16094069251384401>
- Limiany, M., Peranginangin, R. A., & Martinovic, N. (2024). Journal Pre-proof Artificial Intelligence in Open Innovation Project Management: A Systematic Literature Review on Technologies, Applications, and Integration Requirements. *Journal of Open Innovation Technology Market and Complexity*, 13(1), 11. <https://doi.org/10.1016/j.joitmc.2024.100445>
- Dimitrova, M. (2025). Retrieval-Augmented Generation (RAG): Advances and Challenges. *Problems of Engineering Cybernetics and Robotics*, 9(5), 83. <https://doi.org/10.7546/PECR.83.25.03>

- Shahi, T. B., & Sitaula, C. (2022). Natural language processing for Nepali text: a review. *Artificial Intelligence Review*, 55(2), 1-29. <https://doi.org/10.1007/s10462-021-10093-1>
- Giri, S. (2020). E-government Use in Nepal: Issues of Database Management and Data Security. *Journal of the Institute of Engineering*, 15(2), 226-232. <https://doi.org/10.3126/jie.v15i2.27669>
- Al-Fedaghi, S. (2021). UML Sequence Diagram: An Alternative Model. 56(18). <https://doi.org/0.48550/arXiv.2105.15152>
- Tarhini, A., Hussain, A., & Tarhini, T. (2015). Analysis of the Critical Success Factors for Enterprise Resource Planning Implementation from Stakeholders' Perspective: A Systematic Review. *International Business Research*, 8(4), 25-25. <https://doi.org/10.5539/ibr.v8n4p25>

Appendices

Appendix A: Project Submission Form

STUDENT ID	NP069811
INTAKE ID	NP3F2509IT
STUDENT NAME	Ashok Bhattarai

A.. Project Title

"Design and Development of an AI-Powered Cloud-Based Public Notice Management System for Nepal"

B.. Brief description of project background. (i.e. problem context, rationale, description of problem area, nature of challenge)

Problem Context

Public notices and press releases form a critical part of civic communication in Nepal. Government bodies regularly publish examination schedules, job vacancies, tender announcements, policy updates, and official statements. However, these notices are scattered across dozens of individual government websites, each with its own format, layout, and publication mechanism. There is no single platform where a citizen can find all relevant notices in one place.

This situation forces users (particularly students, job seekers, and professionals) to manually visit multiple government portals to stay updated. The process is time-consuming and inefficient. Making it worse, the majority of notices are published in unstructured formats such as long-form text, scanned images, or PDF documents, which are difficult to search through or quickly comprehend. A student looking for an exam schedule or a job seeker checking for vacancy notices often has to wade through lengthy documents just to find a single piece of information.

Nepal currently has no centralized, intelligent system for managing public notices. The absence of such a platform means that official information often fails to reach the people who need it most, simply due to the difficulty of access. This represents a clear gap in Nepal's public information infrastructure that this project aims to address.

Rationale

The advancement of artificial intelligence and cloud computing makes it feasible to build a unified system that solves the fragmentation problem. By using web scraping to automatically collect notices from selected government websites, and applying AI techniques to classify and summarize them, the system can deliver clean, structured, and searchable information to users through a single interface.

This project proposes to design and develop such a system as a Proof of Concept (PoC). The PoC will target 5-6 major Nepal government websites to demonstrate that the approach works in practice. Additionally, an optional Retrieval Augmented Generation (RAG) feature will allow users to upload their own notice-related files such as PDFs or scanned images and ask questions about them in plain language, receiving clear and direct answers without needing to read the entire document.

Tangible Benefits

- Citizens can find public notices from multiple government sources in one place, saving time and effort.

- AI-generated summaries reduce the reading burden so users can understand a notice without reading the full document.
- A searchable and filterable database makes it easy to locate specific notice types quickly.
- The RAG feature allows users to ask questions directly about uploaded documents and get instant answers.

Intangible Benefits

- Reduced information gap between government and the public, especially for rural and less-connected users.
- Greater civic awareness as official information becomes easier to access and understand.
- Increased trust in digital government services through a reliable and intelligent notice platform.
- Empowerment of students and job seekers who depend on timely access to official announcements.

Nature of Challenge

The development of this system comes with several notable challenges. First, government websites in Nepal are highly inconsistent in structure, which makes building reliable scrapers a significant technical task. Second, many notices are published as scanned PDFs or images, requiring Optical Character Recognition (OCR) before any further processing can take place. Third, integrating AI components such as classification, summarization, and RAG into a coherent and responsive system requires careful system design. Fourth, ensuring the scraping process is ethical, by respecting rate limits and not overloading servers, adds another layer of complexity. Finally, maintaining the system's accuracy and relevance as government websites are updated over time will require ongoing monitoring and adjustment.

C.. Brief description of project objectives. (i.e. scope of proposal and deliverables)

Deliverables

The system will provide a cloud-based, AI-powered web platform where users can search, browse, and interact with public notices collected from Nepal government websites. The core functions of the system that will be achieved before delivery are organized as follows:

Admin

- Allow admin to manage and monitor scraping tasks for each government website source.
- Allow admin to review, approve, or remove incorrectly scraped or duplicate notices.
- Allow admin to configure notice categories and AI classification labels.
- Allow admin to manage user access and system settings.
- Allow admin to view system health, scraping logs, and processing reports.

General Users (Students, Job Seekers, Professionals)

- Allow users to search and filter public notices by category, date, keyword, and source website.
- Allow users to view AI-generated summaries of notices without reading the full document.

- Allow users to read the full notice content when needed.
- Allow users to upload their own PDF or image files and ask questions about the content using natural language.
- Allow users to receive clear and accurate answers extracted from their uploaded documents.

System (Automated)

- Automatically scrape public notices from 5-6 selected Nepal government websites on a scheduled basis.
- Automatically extract text from PDF and image-based notices using OCR.
- Automatically classify each notice into a predefined category such as Vacancy, Examination, Tender, Policy Update, or General Announcement.
- Automatically generate a concise summary for each notice and store it alongside the original content.
- Automatically index notice content for fast and accurate search retrieval.

D.. Brief description of the resources needed by the proposal. (i.e. hardware, software, access to information/expertise, user involvement, etc.)

Hardware

The minimum hardware requirements for carrying out the development and meeting the objectives of the system are as follows:

- Processor: Intel Core i5 (or equivalent), 2.4 GHz
- Random Access Memory (RAM): 8 GB
- Storage: 256 GB SSD
- Keyboard, Mouse, and Monitor
- Stable Internet Connection (for cloud access, scraping, and API calls)

Cloud infrastructure resources including virtual machines, managed databases, and object storage will be provisioned through a cloud platform, removing the need for significant on-premise hardware investment.

Software

The minimum software requirements for development and execution of the project are as follows. All tools and platforms selected are free and open-source or available under a free tier:

Backend and API Processing

- Node.js: Runtime environment for executing the backend application (free and open-source)
- NestJS: Structured and scalable backend framework for building the REST API, handling routing, modules, and service logic (free and open-source)
- Python 3.x: Used alongside the NestJS backend for AI and scraping tasks via separate microservice scripts (free and open-source)
- Scrapy and BeautifulSoup4: For building automated web scrapers to collect notices from government websites (free and open-source)
- Selenium with ChromeDriver: For handling JavaScript-rendered pages during scraping where necessary (free and open-source)

- Tesseract OCR: For extracting text from scanned PDFs and image-based notices (free and open-source)
- Hugging Face Transformers: For running open-source NLP models for text classification and summarization (free and open-source)
- Ollama: For running open-source large language models such as LLaMA 3 and Mistral locally, used for the RAG inference feature (free and open-source)
- LangChain: For building the RAG pipeline that connects documents, embeddings, and the language model (free and open-source)
- ChromaDB: For storing and retrieving vector embeddings used in the RAG document interaction module (free and open-source)
- node-cron: For scheduling periodic scraping tasks automatically within the NestJS application (free and open-source)

Frontend

- Next.js: React-based frontend framework for building the user interface and notice dashboard, with support for server-side rendering and static generation (free and open-source)
- Tailwind CSS: For styling the user interface efficiently (free and open-source)
- TypeScript: Typed superset of JavaScript used across both frontend and backend for improved code reliability (free and open-source)

Database and Cloud Deployment

- Supabase: Cloud-based managed PostgreSQL database platform with built-in authentication, storage, and REST API support (free tier available)
- AWS (Amazon Web Services) or Railway: For deploying the NestJS backend and Python scraping services to the cloud. Railway offers a simple free tier for small deployments, while AWS provides a broader free tier with more control (free tier available on both)
- Vercel: Free-tier platform for deploying the Next.js frontend (free tier available, optimized for Next.js)
- Docker: For containerizing application components to ensure consistent and portable deployment (free and open-source)
- Git and GitHub: For version control and source code management (free)

Documentation and Planning

- Microsoft Word: For writing project documentation and reports
- draw.io: Free browser-based tool for creating system architecture and data flow diagrams (free)
- Visual Studio Code: As the primary code editor (free)
- Notion: For project planning, task tracking, and sprint management (free tier available)

Access to Information/Expertise

Consultation and guidance will be sought from academic supervisors who have expertise in artificial intelligence, cloud computing, and software engineering. Technical documentation from Supabase, AWS, Railway, and Vercel, as well as library and framework documentation for NestJS, Next.js, Hugging Face, LangChain, Ollama, and Scrapy, will be studied and applied throughout the project. Research papers on NLP, web scraping, and public information systems will also be reviewed. Public government websites of Nepal will serve as the primary data source.

User Involvement

The system will be developed with the needs of its target users in mind, particularly students, job seekers, and general citizens who rely on timely access to government notices. I will gather feedback from these stakeholder groups during the usability testing phase to evaluate how effectively the interface meets their needs. Their input will be used to improve the search functionality, notice presentation, and AI summary quality before final delivery.

E.. Academic research being carried out and other information, and techniques being learned. (i.e. what are the names of books you are going to read / data sets you are going to use)

To carry out the deliverables, the preliminary list of books and online resources I will study are as follows:

Books

Name: Web Scraping with Python: Collecting More Data from the Modern Web

Author: Ryan Mitchell

Publisher: O'Reilly Media

Name: Natural Language Processing with Transformers

Author: Lewis Tunstall, Leandro von Werra, Thomas Wolf

Publisher: O'Reilly Media

Name: Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3rd Edition)

Author: Aurélien Géron

Publisher: O'Reilly Media

Name: Cloud Computing: Concepts, Technology and Architecture

Author: Thomas Erl, Zaigham Mahmood, Ricardo Puttini

Publisher: Prentice Hall

Name: Flask Web Development: Developing Web Applications with Python (2nd Edition)

Author: Miguel Grinberg

Publisher: O'Reilly Media

Online Resources

- Minaee et al., "Deep Learning Based Text Classification: A Comprehensive Review", ACM Computing Surveys, Volume 54
- Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", Advances in Neural Information Processing Systems (NeurIPS 2020)
- Wirtz, Weyerer & Geyer, "Artificial Intelligence and the Public Sector: Applications and Challenges", International Journal of Public Administration, Volume 42
- Janssen, Charalabidis & Zuiderwijk, "Benefits, Adoption Barriers and Myths of Open Data and Open Government", Information Systems Management, Volume 29
- Mell & Grance, "The NIST Definition of Cloud Computing", NIST Special Publication 800-145

F.. Brief description of the development plan for the proposed project. (i.e. which software methodology and why, the major areas of functions to be developed, and the order in which developed)

System Development Methodology

A system development methodology is the framework used to structure, plan, and control the process of developing an information system. For this project, the Agile methodology with an iterative and incremental approach has been selected as the most appropriate development framework.

Agile is well-suited for this project because the system involves multiple interconnected components (web scraping, AI processing, cloud deployment, and a user interface) each of which may require refinement as development progresses. Agile allows each component to be built, tested, and improved in short development cycles rather than committing to a rigid sequential plan. This reduces the risk of discovering major issues late in the project. Furthermore, the iterative nature of Agile makes it easier to incorporate feedback from testing and supervision into each development cycle, ensuring that the final product closely meets the intended objectives.

The development will be organized into the following phases in order:

- Phase 1: Planning and Architecture: Define final system requirements, design the overall system architecture, and set up the development environment including the GitHub repository, Supabase database, NestJS backend scaffold, and Next.js frontend scaffold.
- Phase 2: Web Scraping Module: Develop and test automated web scrapers for each of the 5-6 selected Nepal government websites using Scrapy and BeautifulSoup.
- Phase 3: Data Storage and Processing Pipeline: Design the cloud database schema and implement the pipeline for storing and organizing scraped notice data.
- Phase 4: AI Classification and Summarization: Integrate NLP models for automatically categorizing notices and generating concise summaries.
- Phase 5: Web Interface: Build the Next.js frontend dashboard where users can search, filter, and read notices, connected to the NestJS backend API.
- Phase 6: RAG Document Interaction Module: Implement the optional feature for uploading documents and asking questions in natural language using LangChain and a language model API.
- Phase 7: Testing and Deployment: Conduct all testing phases, fix identified issues, and deploy the final system with the Next.js frontend on Vercel, the NestJS backend on AWS or Railway, and the database on Supabase.
- Phase 8: Documentation: Prepare the full project report, system documentation, and user guides.

G.. Brief description of the evaluation and test plan for the proposed project. (i.e. what are the success criteria and how will be evaluated and implementation will be tested, indicate the estimated size of the demonstration/test database)

Success Criteria

The main goal of the AI-Powered Cloud-Based Public Notice Management System for Nepal will be considered achieved if the following criteria are met:

The system must reliably scrape and store public notices from at least 5 Nepal government websites without errors. The AI classification module must correctly categorize notices with reasonable accuracy across the defined categories: Vacancy, Examination, Tender, Policy Update, and General Announcement. AI-generated summaries must be coherent, accurate, and meaningfully shorter than the original notice content. The web interface must allow users to easily search and find relevant notices. The optional RAG module must return relevant and

correct answers when users ask questions about uploaded notice documents. To ensure the system is working correctly before delivery, the following testing strategies will be applied:

Unit Testing

In unit testing, each individual component of the system will be tested in isolation to verify that it works correctly on its own. The web scraper for each government website will be tested separately to confirm it extracts the correct data. The AI classification and summarization components will be tested using a set of sample notices to verify that outputs match expected results. Each API endpoint will be tested individually using tools such as Postman to confirm correct request handling and response formatting. For example, the classification module should correctly label a vacancy notice as "Vacancy" and an exam date notice as "Examination" without confusion between categories.

Integration Testing

Once individual units pass testing, they will be combined and tested together to ensure all components work as a complete system. Integration testing will verify that notices scraped from government websites are correctly passed through the processing pipeline, stored in the cloud database, and accurately retrieved and displayed by the web interface. For example, when the scraper collects a new notice, the AI pipeline should automatically classify and summarize it, and a user searching for that notice through the web interface should be able to find and read it without any data loss or formatting issues.

Usability Testing

Usability testing will involve real users from the target audience (students, job seekers, and general citizens) interacting with the system's web interface under realistic conditions. Participants will be asked to complete specific tasks such as finding a vacancy notice, reading its AI summary, and using the document upload feature to ask a question. Feedback will be collected through structured questionnaires evaluating ease of use, clarity of information, search accuracy, and overall satisfaction. The results will be used to make final improvements to the interface before project delivery.

Test Database

The demonstration and test database will contain approximately 200 to 300 sample public notices collected from the targeted Nepal government websites. These notices will cover all defined categories (Vacancy, Examination, Tender, Policy Update, and General Announcement) to ensure that classification, summarization, and search functionality are tested across the full range of content types the system is expected to handle in production.

Appendix B: Project Proposal Form



Name: Ashok Bhattarai

Supervisor: Prof. Dr. RN Thakur, Mr. Rajan Raj Pant, Mr. Shambhu Gautam

Student Number: NP069811

Intake: NP3F2509IT

Email: ashok.bhattarai@study.lbef.edu.np

Program Name: B. Sc. (Hons.) Information Technology

Project Title: Design and Development of an AI-Powered Cloud-Based Public Notice Management System for Nepal

Introduction

The press releases and public notices are very critical sources of information for citizens, students, professionals, and organizations alike, as they indicate examinations, vacancies, tenders, policy updates, and official statements. It is very difficult, however, for this information to be found in the multitude of government websites, each in a different format and structure, and with different means of publication.

This indeed compels users to go to these fragmented sites themselves. Being a traditional and slow way of getting updated, it is quite inefficient in its nature. Majority of the notices come in unstructured format: this may be quite often a long text, a scanned document, or even in a PDF format, which is hard to search, analyze, or readily comprehend. This is a huge headache for students and job seekers who need timely access to official information.

Artificial intelligence and cloud computing these are the two buzzwords that have been hogging the limelight in this 2021. They make it feasible to come up with a unified system that would aggregate public notices and even allow for intelligent interaction based on the notice's content. This project is about the design and development of Nepal-based intelligent public notice management system enabled with AI and cloud computing. This was the Proof of Concept (PoC) implementation when the system briefly outlines how it collects and processes notices and press releases from a few major government websites for the prototype system (Wirtz et al., 2019). The system was developed to be scalable, meaning more government websites could be incorporated in the future.

On retrieval augmented generation, the system also allows an optional feature. Users will be able to upload their notice-related files in the form of PDF or Image and question the system about them in natural language. This further enhances the accessibility of the user in question-answering interaction with complex or lengthy notices.

Problem Statement

In the case of Nepal, despite the existence of government platforms, public notices and press releases are highly fragmented across numerous websites with no centralized or intelligent access to information. A user has to scuttle themselves through vast documents in which key details are usually missed either because of time constraints or simply a lack of clarity (Janssen et al., 2012).

Secondly, the existing systems do not possess interactivity features, which would make it easy for users to understand the notice content in a complex or unstructured format such as PDFs or scanned images. There isn't even a single cloud-based platform in Nepal today that automatically collects notices and processes them using artificial intelligence techniques while engaging users by asking them questions where required. This gap creates a big problem in effective public information dissemination (Cui et al., 2021).

Aim

The project aims at designing and developing a cloud-based, AI-powered public notice management solution. This solution will be set to capture public notices and press releases of the Government of Nepal from the specified websites, proof the concept, with optional AI-based interactive features for better information access.

Objectives

- To design and develop a cloud-based system for centralized management of public notices and press releases.
- To implement web scraping mechanisms to collect notices from selected Nepal government websites as a Proof of Concept.
- To apply artificial intelligence techniques for classification and summarization of notice content.
- To store and manage structured notice data using cloud-based databases.
- To develop a user-friendly web interface for searching and monitoring public notices.

- To implement an optional AI-based document interaction feature that enables users to upload PDF or image files and obtain answers through natural language queries.

Literature Review

Literature Review 1: Web Scraping for Public Information Systems

Web scraping is arguably considered one of the most intricate forms of data extraction in this new era of collecting information that is widely distributed and publicly flowing across the web regarding those cases where the data publishers do not avail highly structured APIs.

It is within this line that an attempt by Mitchell in 2018 can be traced to making a comprehensive exposition on the methodologies of web scraping, majorly focusing on technical implementation and ethical considerations involved. This is done so that the information is systematized and automatic in the process of capturing HTML tips of websites, by using parsing techniques, DOM traversal, and pattern recognition algorithms.

Some of the major principles expounded in Mitchell's work about responsible web scraping would be adherence to robots.txt conventions, the proper implementation of rate limiting so no server is overwhelmed during the scraping activity, using headless browsers to manage dynamic content only when necessary, and full adherence to terms of service and legal frameworks. Practical methods were presented for some common problems in pagination, session state management, processing the content rendered by JavaScript, and strong error handling techniques.

It is also the argument of Mitchell in his research that automated data extraction increases the availability of information by changing it from the vast and diverse world of web-based content to structured datasets that can be analyzed, searched, and presented in user-friendly ways. This saves a lot of manual work from end users who would have to visit multiple websites individually. Research was carried out in order to debug the information that, if implemented ethically and technically, web scraping acts as a very good remedy in collecting distributed public information. This research systematically supports the feasibility of using web scraping to collect government notices and press releases of the Nepal governments from a number of their websites. In most cases structured APIs are not available. Mitchell's techniques and the ethical framework that has been outlined shall provide the basic guidance

elements on how best to implement the data collection part of this project in a responsible and sustainable manner (Mitchell & R., 2018).

Literature Review 2: AI-Based Text Classification and Summarization

Deep learning architectures have brought a colossal leap to text analysis and natural language processing for artificial intelligence applications. (Minaee et al., 2021) provided a very detailed comparison of deep learning methods for text classification by tracing the evolution from classical machine learning methods up to modern transformer-based models. The survey included a huge number of neural network architectures such as CNNs for text, RNNs with LSTM cells, attention mechanisms, and state-of-the-art transformer models like BERT and GPT. This study goes on to prove that deep learning technologies can automatically learn hierarchical presentations of textual data that will make it easy for documents to be correctly classified into earlier defined categories without involving a lot of manual feature engineering.

For government notices and press releases, such a capability means that the system would be able to automatically categorize them into notification of examination schedules, job vacancies and tender publications, updates in policies, and ideological shifts, and official announcements. The first one is carried out in the sense of selecting important sentences from the documents and creating condensed versions with the essential meanings retained.

They find that this neural summarization model can always generate coherent and informative summaries, and at the same time, represent the critical information in the documents while much smaller in size. Automatic summary generation of bulky representations from government notices and press releases would thus help users get exposed to vital information much more quickly than having to go through the unending process of handbook discovery. The essence of this research infers that usability via AI-based natural language processing for unstructured textual data enhances significantly to make it intelligently content organized, quick, and efficient information retrieval, and enhancement in understanding through summarization.

These are directly linked with the aims of this project, which is to classify and summarize government notices to make it easy to access information for any concerned citizens, students, and professionals who need to be up-to-date on government announcements.

Literature Review 3: Cloud-Based Information Management Systems

Cloud computing has since altered how information systems are conceived, implemented, and scaled by allowing organizations to realize computing resources without upfront investments in physical infrastructure. Among the important characteristics of cloud computing are on-demand resource provisioning, broad network access, resourced pooling via shared infrastructure, rapid elasticity, and pay-per-use pricing models. Information Management Systems will gain really big from architectures based on the cloud (Mell et al., 2011).

Scalability allows systems to manage increased workloads without major redesigns while flexibility helps developers to choose the right deployment model and service by meeting system requirements and being cost-effective through reduced investment needs in hardware and operation overheads. The Google Cloud Platform helps improve trustworthiness and availability with built-in redundancy for systems, backup, and disaster recovery support.

In the instant project, the benefits ideally map to the requirements with the public notice management information system, which is now available in the form of a scalable prototype initiated as proof of concept starting from the selected websites belonging to the Government of Nepal. Being cloud deployable, the existence of further data sources provides the gradual expansion opportunities without hurting accessibility, maintainability, and readiness for further enhancements onto more advanced AI features or increased user demand.

Deliverables

1. AI-Powered Cloud-Based Public Notice Management System

A functional web-based system deployed on a cloud platform that centrally manages public notices and press releases from selected Nepal government websites.

2. Proof of Concept (PoC) Implementation

A working demonstration of automated notice collection from 5–6 major Nepal government websites, validating the feasibility of the proposed solution.

3. Scalable System Architecture

A modular and cloud-ready system design that supports future expansion to additional government websites and data sources.

4. **AI-Based Notice Processing Module**

Artificial intelligence components for automatic classification and summarization of public notices and press releases.

5. **Centralized Cloud Database**

A structured cloud-based database for storing notice metadata, full content, summaries, and classification labels.

6. **Web-Based User Interface**

An interactive dashboard that allows users especially students and job seekers to search, filter, and monitor public notices efficiently.

7. **Optional RAG-Based Document Interaction Feature**

An AI-powered module that enables users to upload PDF or image documents and ask questions in natural language, receiving simplified and relevant answers.

8. **System Documentation and Final Project Report**

Comprehensive documentation covering system design, methodology, implementation, evaluation, and future enhancement possibilities.

References

- Cui, L., Xu, Y., L., T., Wei, & F. (2021). Document AI: Benchmarks, Models and Applications. *arXiv preprint*.
- Janssen, M., Charalabidis, Y., & Zuiderwijk. (2012). Benefits, Adoption Barriers and Myths of Open Data and Open Government. *Information Systems Management*, 258-268.
- Mell, P., Grance, & T. (2011). The NIST Definition of Cloud Computing. *NIST Special Publication*.
- Minaee, S., Kalchbrenner, N., Cambria, E., Nikzad, N., Chenaghlu, . . . Gao, J. (2021). Deep Learning Based Text Classification: A Comprehensive Review. *ACM Computing Surveys*, 1-40.
- Mitchell, & R. (2018). Web Scraping with Python: Collecting More Data from the Modern Web.
- Wirtz, W., B., Weyerer, C., J., & Geyer. (2019). Artificial Intelligence and the Public Sector—Applications and Challenges. *International Journal of Public Administration*, 596-615.

Appendix C: Ethics Forms

Office Record	Receipt - Fast-Track Ethical Approval, Student name: <u>Ashok Bhattacharya</u>
Date Received:	Student number: <u>AP065511</u>
Received by whom:	Received by: <u></u> Date: <u>26th Feb 2026</u>

APU / APIIT FAST-TRACK ETHICAL APPROVAL FORM (STUDENTS)

Tick one box (level of study): ☐ POSTGRADUATE (PhD / MPhil / Masters) ☒ UNDERGRADUATE (Bachelors degree) ☐ FOUNDATION / DIPLOMA / Other categories	Tick one box (purpose of approval): ☒ Thesis / Dissertation / FYP project ☐ Module assignment ☐ Other: _____
Title of Programme on which enrolled <u>B.Sc. IT (IE)</u>	
Tick one box: ☒ Full-Time Study or ☐ Part-Time Study	
Title of project / assignment <u>Design and development of an AI-powered cloud-based Public Notice Management System for Nepal</u>	
Name of student researcher <u>Ashok Bhattacharya</u>	
Name of supervisor / lecturer <u>Dr. Laxman Mandal</u>	

Student Researchers- please note that certain professional organisations have ethical guidelines that you may need to consult when completing this form.

Supervisors/Module Lecturers - please seek guidance from the Chair of the APU Research Ethics Committee if you are uncertain about any ethical issue arising from this application.

		YES	NO	N/A
1	Will you describe the main procedures to participants in advance, so that they are informed about what to expect?	☒		
2	Will you tell participants that their participation is voluntary?	☒		
3	Will you obtain written consent for participation?	☒		
4	If the research is observational, will you ask participants for their consent to being observed?	☒		
5	Will you tell participants that they may withdraw from the research at any time and for any reason?	☒		
6	With questionnaires and interviews will you give participants the option of omitting questions they do not want to answer?	☒		
7	Will you tell participants that their data will be treated with full confidentiality and that, if published, it will not be identifiable as theirs?	☒		
8	Will you give participants the opportunity to be debriefed i.e. to find out more about the study and its results?	☒		

If you have ticked **No** to any of Q1-8 you should complete the full Ethics Approval Form.

		YES	NO	N/A
9	Will your project/assignment deliberately mislead participants in any way?		☒	
10	Is there any realistic risk of any participants experiencing either physical or psychological distress or discomfort?		☒	
11	Is the nature of the research such that contentious or sensitive issues might be involved?		☒	

If you have ticked **Yes** to 9, 10 or 11 you should complete the full Ethics Approval Form. In relation to question 10 this should include details of what you will tell participants to do if they should experience any problems (e.g. who they can contact for help). You may also need to consider risk assessment issues.

		YES	NO	N/A
12	Does your project/assignment involve work with animals?		☒	
13	Do participants fall into any of the following special groups? Note that you may also need to obtain satisfactory clearance from the relevant authorities		☒	
	Children (under 18 years of age)			
	People with communication or learning difficulties			
	Patients			
	People in custody			
	People who could be regarded as vulnerable			
	People engaged in illegal activities (eg drug taking)			
14	Does the project/assignment involve external funding or external collaboration where the funding body or external collaborative partner requires the University to provide evidence that the project/assignment had been subject to ethical scrutiny?		☒	

If you have ticked Yes to 12, 13 or 14 you should complete the full Ethics Approval Form. There is an obligation on student and supervisor to bring to the attention of the APU Research Ethics Committee any issues with ethical implications not clearly covered by the above checklist.

STUDENT RESEARCHER

Provide in the boxes below (plus any other appended details) information required in support of your application, THEN SIGN THE FORM.

Please Tick Boxes

I consider that this project/assignment has no significant ethical implications requiring a full ethics submission to the APU Research Ethics Committee.	☒
Give a brief description of participants and procedure (methods, tests used etc) in up to 150 words. Participants will be drawn from the intended target user groups of the system, mainly students, jobseekers, and general citizens who access public notices from government bodies in Nepal. No vulnerable groups will be specifically targeted. The procedure involves usability and functional evaluation of the developed prototype. Participants will complete predefined tasks such as searching and filtering notices, viewing AI-generated summaries, accessing full content and using the document upload and question-answering feature. Data will be collected through a short post-task questionnaire and brief feedback sessions to assess usability and clarity of system.	
I also confirm that: i) All key documents e.g. consent form, information sheet, questionnaire/interview are appended to this application.	☒
Or ii) Any key documents e.g. consent form, information sheet, questionnaire/interview schedules which need to be finalised following initial investigations will be submitted for approval by the project/assignment supervisor/module lecturer before they are used in primary data collection.	☒

E-signature:  Print Name: Ashok Bhattacharya Date: 26th Feb 2026
(Student Researcher)

Please note that any variation to that contained within this document that in any way affects ethical issues of the stated research requires the appending of new ethical details. New ethical consent may need to be sought.

The completed form (and any attachments) should be submitted for consideration by your Supervisor/Module Lecturer

SUPERVISOR/MODULE LECTURER
PLEASE CONFIRM THE FOLLOWING:

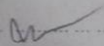
Please Tick Box

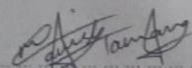
I consider that this project/assignment has no significant ethical implications requiring a full ethics submission to the APU Research Ethics Committee	☒
i) I have checked and approved the key documents required for this proposal (e.g. consent form, information sheet, questionnaire, interview schedule)	☒
Or	
ii) I have checked and approved draft documents required for this proposal which provide a basis for the preliminary investigations which will inform the main research study. I have informed the student researcher that finalised and additional documents (e.g. consent form, information sheet, questionnaire, interview schedule) must be submitted for approval by me before they are used for primary data collection.	

SUPERVISOR AND SECOND ACADEMIC SIGNATORY

STATEMENT OF ETHICAL APPROVAL (please delete as appropriate)

- 1) THIS PROJECT/ASSIGNMENT HAS BEEN CONSIDERED USING AGREED APU/SU PROCEDURES AND IS NOW APPROVED
- 2) THIS PROJECT/ASSIGNMENT HAS BEEN APPROVED IN PRINCIPLE AS INVOLVING NO SIGNIFICANT ETHICAL IMPLICATIONS, BUT FINAL APPROVAL FOR DATA COLLECTION IS SUBJECT TO THE SUBMISSION OF KEY DOCUMENTS FOR APPROVAL BY SUPERVISOR (see Appendix A)

E-signature.....  Print Name Dr. Karmen Mandab Date 26/02/2026
(Supervisor/Lecturer)

E-signature.....  Print Name Manish Kumar Taurang Date 1st March 2026
(Second Academic Signatory)

Office Record	Receipt - Appendix A (Fast-Track Ethics Form)
Date Received	Student name: <u>Ashok Bhattarai</u>
Received by whom	Student number: <u>NP069811</u>
	Received by: <u>Dr. Laxman Mandabate</u>
	Date: <u>24th Feb 2026</u>

APPENDIX A AUTHORISATION FOR USE OF KEY DOCUMENTS

Completion of Appendix A is required when for good reasons key documents are not available when a fast track application is approved by the supervisor/module lecturer and second academic signatory.

I have now checked and approved all the key documents associated with this proposal e.g. consent form, information sheet, questionnaire, interview schedule

Title of project/assignment... Design and Development of AI-powered Cloud-based Public Notice Management System for Nepal

Name of student researcher... Ashok Bhattarai

Student ID: NP069811 Intake: NP3F2509 IT (LE)

E-signature... [Signature] Print Name: Dr. Laxman Mandabate Date: 26/02/2026
(Supervisor/Lecturer)

Appendix D: Supervisor Meeting Log Sheets

Project Log Sheet – Supervisory Session

Notes on use of the project log sheet

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum SIX(6) during the course of the project (SIX mandatory supervisory sessions)
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisor session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
5. The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
6. It is recommended that students bring along log sheets of previous meetings together with the project file during each supervisory session.
7. The log sheet is an important deliverable for the project and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's Name: Ashok Bhattacharzi Date: 26/02/2026 Meeting No: 1

Project Title: Design and Development of an AI-powered Cloud-based Public Notice Management System for Nepal Intake: NP 3F 2509 IT (E)

Supervisor's Name: Dr. Laxman Mandal Signature: [Signature]

Items for discussion (noted by student before mandatory supervisory meeting):

1. To introduce the project topic and overall project idea.
2. To explain what the project is about and its main purpose.
3. To present the dummy website prototype.
4. To discuss the planned features of the system.
5. To clarify the project objectives and scope.

Record of discussion (noted by student during mandatory supervisory meeting):

1. I introduced my project topic and explained the overall concept.
2. I showed the dummy website prototype and explained the basic layout and features.
3. I explained what I am developing and how the system works.
4. The supervisor advised me to align the project properly with clear objectives.
5. The supervisor emphasized that the objectives must match the project outcomes for making.

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

1. Rewrite and refine the project objectives clearly.
2. Ensure all objectives match the system features and scope.
3. Improve the dummy website prototype based on feedback.
4. Update the project description and scope.
5. Prepare progress and materials for the next meeting.

SUPERVISOR COPY

Project Log Sheet – Supervisory Session

Notes on use of the project log sheet:

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum SIX (6) during the course of the project (SIX mandatory supervisory sessions).
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
5. The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
6. It is recommended that students bring along log sheets of previous meetings together with the project file during each supervisory session.
7. The log sheet is an important deliverable for the project and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's Name: Ashok Bhattarai Date: 16/03/2026 Meeting No: 2

Project Title: Design and development of an AI-powered Chat-based Public Notice Management System for Nepal Intake: NP352509IT

Supervisor's Name: Dr. Laxman Hazudai Signature: [Signature]

Items for discussion (noted by student before mandatory supervisory meeting):

1. To present the revised project objectives based on supervisor feedback.
2. To discuss the web scraping approach for collecting government notices.
3. To discuss the implementation plan for the RAG document interaction module.
4. To explain the proposed system architecture and technology stack.
5. To explain the AI processing pipeline for OCR, classification and summarization.

Record of discussion (noted by student during mandatory supervisory meeting):

1. The revised project objectives were reviewed and finalized.
2. The overall system idea and architecture were discussed.
3. The selected technology stack was reviewed and confirmed.
4. The web scraping approach for government websites was discussed.
5. The supervisor suggested starting with basic modules before advanced features.

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

1. Prepare a detailed project document based on discussion.
2. Finalize system architecture diagram.
3. Select and confirm government websites for scraping.
4. Do basic research on scraping tools and methods.
5. Plan the next phase of implementation.

SUPERVISOR COPY

Project Log Sheet – Supervisory Session

Notes on use of the project log sheet:

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum SIX(6) during the course of the project (SIX mandatory supervisory sessions).
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisor session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
5. The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
6. It is recommended that students bring along log sheets of previous meetings together with the project file during each supervisory session.
7. The log sheet is an important deliverable for the project and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's Name: <u>Ashok Bhattarai</u>	Date: <u>6th April 2026</u>	Meeting No: <u>3</u>
Project Title: <u>Design and development of AI-powered cloud-based Public Notice Management System for Nepal</u>		Intake: <u>NIP3F2SC9.I.T</u>
Supervisor's Name: <u>Dr. Karmann Nandall</u>		Signature: <u>[Signature]</u>
Items for discussion (noted by student <u>before</u> mandatory supervisory meeting): 1. To present the prepared updated project document. 2. To explain the finalized system architecture diagram. 3. To discuss the selected government websites for scraping. 4. To present initial research on web scraping tools and methods. 5. To discuss the plan in for starting implementation.		
Record of discussion (noted by student during mandatory supervisory meeting): 1. The project document was reviewed and feedback was provided. 2. The system architecture diagram was discussed and refined. 3. The selected government websites were approved. 4. The scraping approach and tools were discussed in detail. 5. The supervisor suggested starting basic implementation of scraping module.		
Action List (to be attempted or completed by student by the <u>next</u> mandatory supervisory meeting): 1. Start basic implementation of web scraping for one website. 2. Improve the project document based on feedback. 3. Set up environment (backend and tools). 4. Begin designing database structure. 5. Prepare progress demonstration for next meeting.		

SUPERVISOR COPY

Project Log Sheet – Supervisory Session

Notes on use of the project log sheet

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum SIX(6) during the course of the project (SIX mandatory supervisory sessions).
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form.
5. The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
6. It is recommended that students bring along log sheets of previous meetings together with the project file during each supervisory session.
7. The log sheet is an important deliverable for the project and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively.

Student's Name: Ashok Bhattarai Date: 20th April 2016 Meeting No: 4

Project Title: Design and Development of AI-powered Cloud Based Public Notice Management System for Nepal Intake: NP 3F2 509 IT

Supervisor's Name: Dr. Karmar Mandal Signature: [Signature]

Items for discussion (noted by student before mandatory supervisory meeting):

1. To present initial progress on web-scraping implementation.
2. To demonstrate scraping results from the selected websites.
3. To explain the development environment setup and tools used.
4. To discuss the initial database design for storing notices.
5. To discuss challenges faced during scraping and possible improvements.

Record of discussion (noted by student during mandatory supervisory meeting):

1. Initial scraping progress and results were presented and reviewed.
2. The scraping approach was discussed and minor improvements were suggested.
3. The development environment setup was confirmed.
4. The database design was discussed and refined.
5. The supervisor advised continuing scraping and improving data handling.

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

1. Improve and expand web scraping for additional websites.
2. Refine the database structure based on feedback.
3. Start integrating scraped data into the Backend system.
4. Handle errors and improve data extraction accuracy.
5. Prepare an improved demonstration for the next meeting.

SUPERVISOR COPY

Project Log Sheet – Supervisory Session

Notes on use of the project log sheet

1. This log sheet is designed for meetings of more than 15 minutes duration, of which there must be at minimum SIX(6) during the course of the project (SIX mandatory supervisory sessions)
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session, and noting these in the relevant sections of the form, effectively forming an agenda for the session
3. A log sheet is to be brought by the STUDENT to each supervisor session
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant section of the form
5. The student should leave a copy (after the session) of the Project Log Sheet with the supervisor and to the administrator at the academic counter. A copy is retained by the student to be filed in the project file.
6. It is recommended that students bring along log sheets of previous meetings together with the project file during each supervisory session
7. The log sheet is an important deliverable for the project and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the final year documentation, with sheets dated and numbered consecutively

Student's Name: Ashok Bhattacharzi Date: 27th April 2020 Meeting No: 5

Project Title: Design and Development of AI-powered Cloud based Public Notice Management System for Nepal Intake: NP3F2S09IT (CE)

Supervisor's Name: Dr. Lakman Mandal Signature: [Signature]

Items for discussion (noted by student before mandatory supervisory meeting):

1. To present the finalized project document for submission.
2. To show initial progress on system setup and web scraping.
3. To review system architecture and planned modules.
4. To discuss any final corrections required in the document.
5. To confirm readiness for document submission.

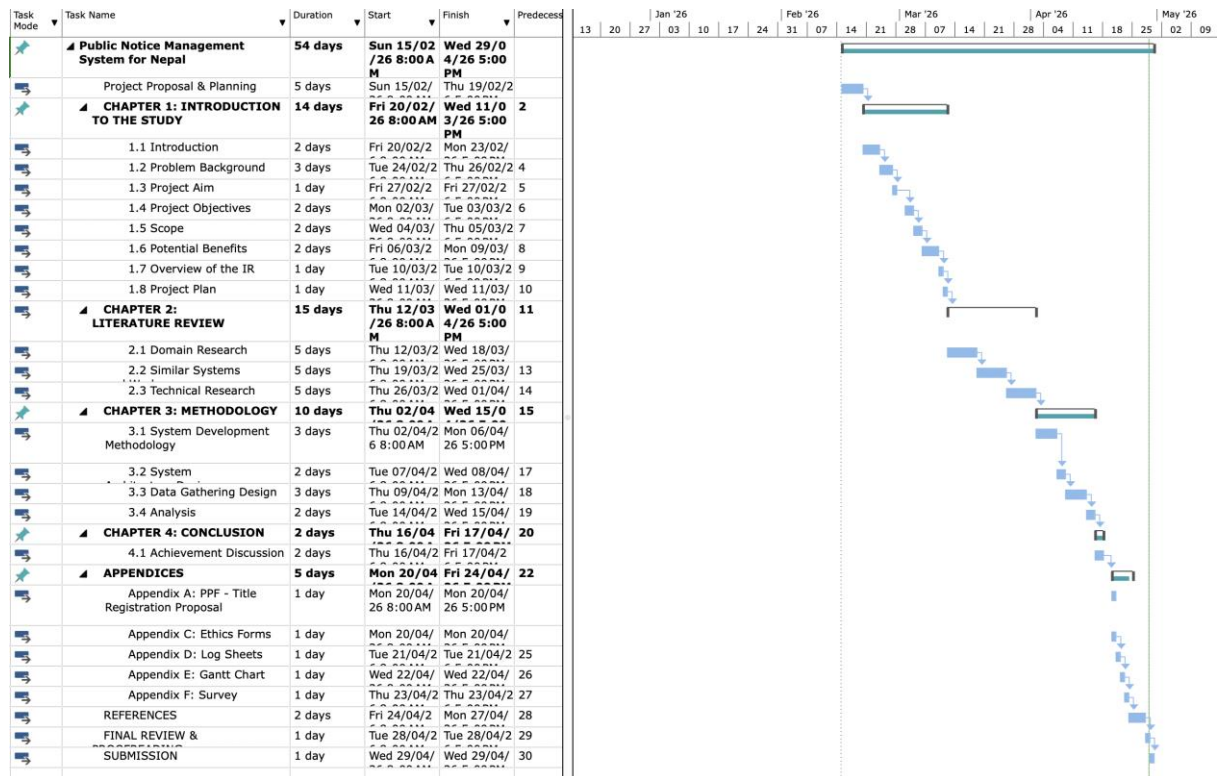
Record of discussion (noted by student during mandatory supervisory meeting):

1. The project document was reviewed and minor improvements were suggested.
2. Initial system setup and scraping progress were discussed.
3. The system architecture and planned modules were confirmed.
4. Feedback was provided to refine the documentation.
5. The supervisor approved the document for submission with minor changes.

Action List (to be attempted or completed by student by the next mandatory supervisory meeting):

1. Make final corrections to the project document
2. Ensure proper formatting and referencing.
3. Finalize diagrams and structure in the report.
4. Submit the finalized Docx document
5. Continue system development in the next semester.

SUPERVISOR COPY



Appendix F: Respondent Demographic Profile

Name:

72 responses

Gita Lamsal

Anil Gautam

Ramesh Bhandari

Aanya Shah

Lata Pradhan

Sneha Baral

Ashok Adhikari

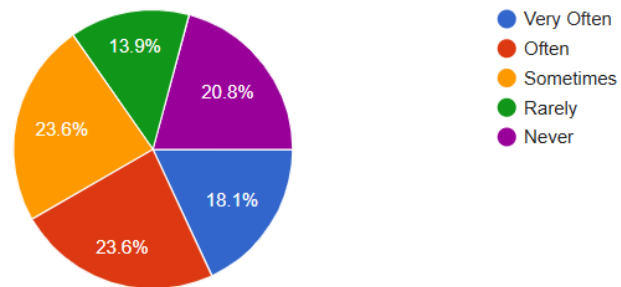
Riya Lamsal

Neha Thapa

1. How often do you visit government websites to check public notices?

[Copy chart](#)

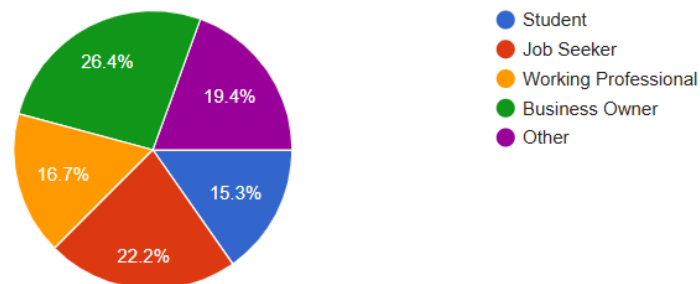
72 responses



2. What is your role?

[Copy chart](#)

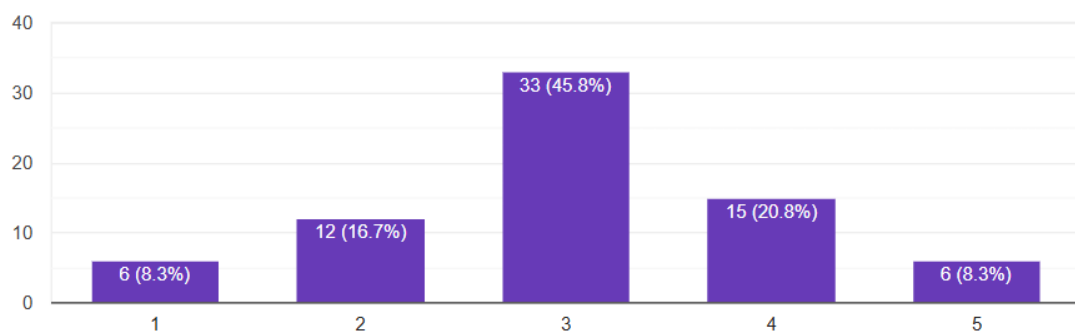
72 responses



3. How satisfied are you with the current system of accessing government notices?

[Copy chart](#)

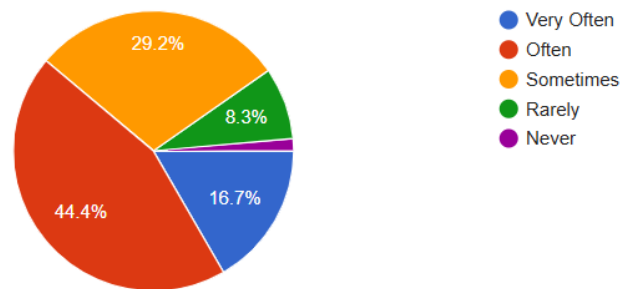
72 responses



4. How often do you face difficulty finding important government notices?

 [Copy chart](#)

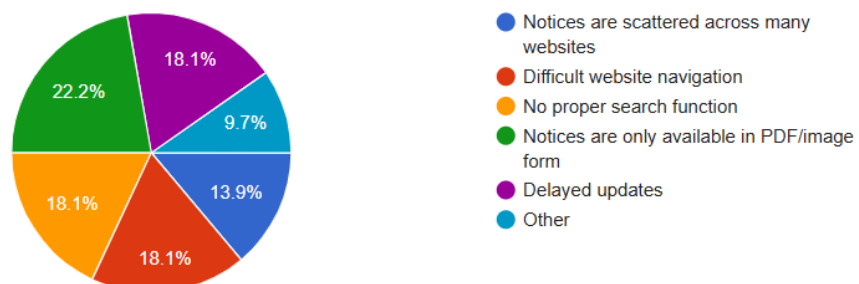
72 responses



5. Which problem do you face most while searching for public notices?

 [Copy chart](#)

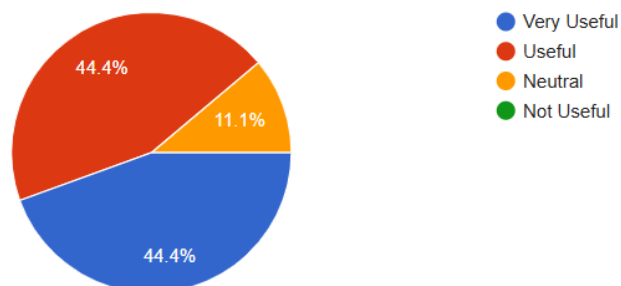
72 responses



6. How useful would a single platform for all government notices be?

 [Copy chart](#)

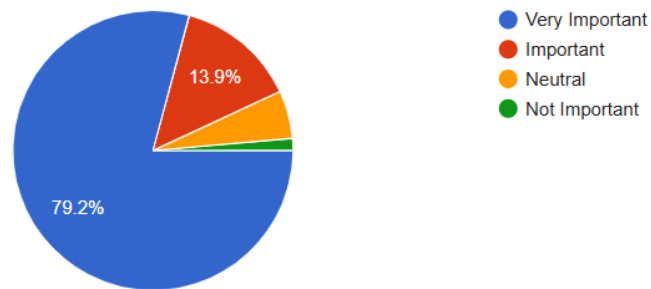
72 responses



7. How important is quick access to job vacancy notices and exam schedules?

 [Copy chart](#)

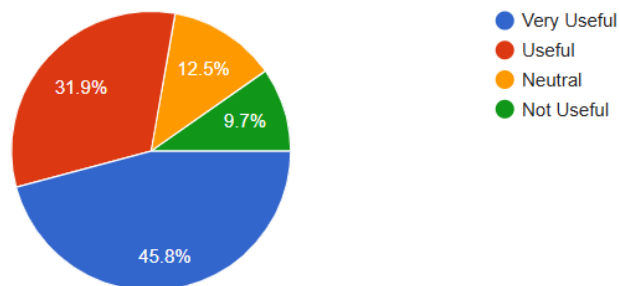
72 responses



8. How useful would AI-generated summaries of long government notices be?

 [Copy chart](#)

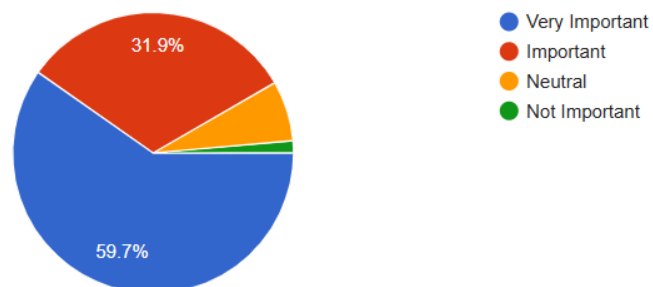
72 responses



9. How important are notifications and alerts for newly published notices?

 [Copy chart](#)

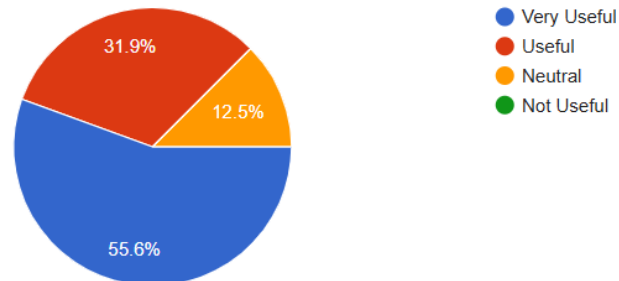
72 responses



10. How useful would category-based filtering (jobs, tenders, exams, policies) be?

 Copy chart

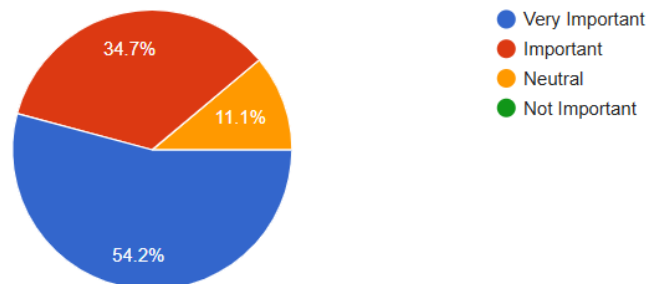
72 responses



11. How important is secure login and user access control?

 Copy chart

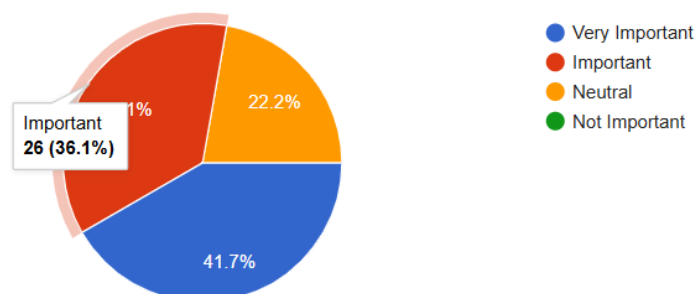
72 responses



12. How important is cloud-based data storage for saving notices and user preferences?

 Copy chart

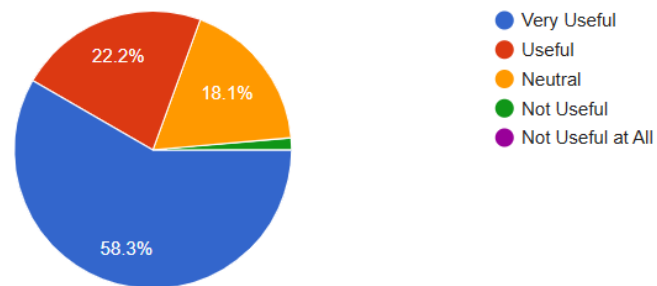
72 responses



13. How useful would keyword-based search functionality be?

 Copy chart

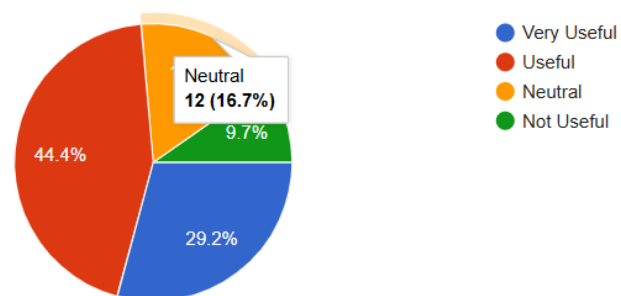
72 responses



14. How useful would document upload and question-answering (AI chatbot) be?

 Copy chart

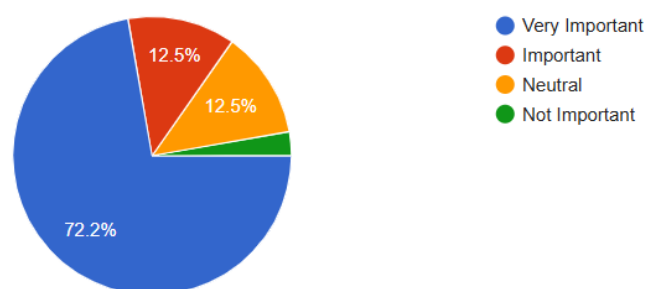
72 responses



15. How important is mobile-friendly access for this platform?

 Copy chart

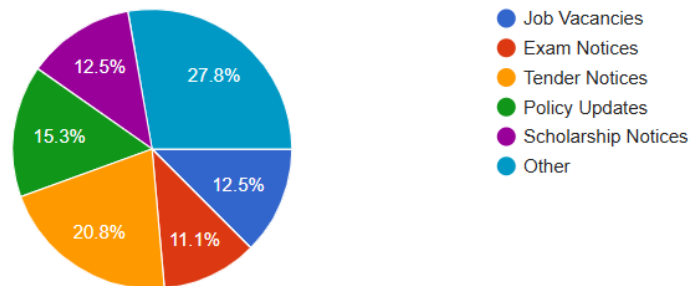
72 responses



16. Which notice category do you search most often?

[Copy chart](#)

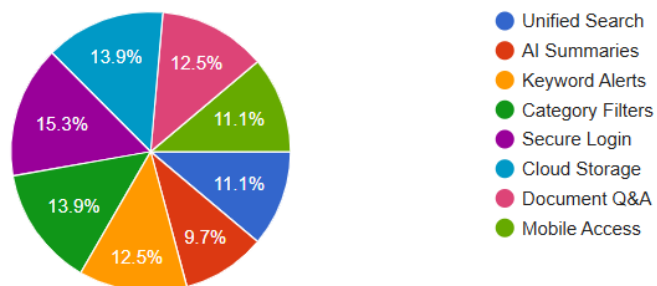
72 responses



17. Which features would you prefer in a public notice management system?

[Copy chart](#)

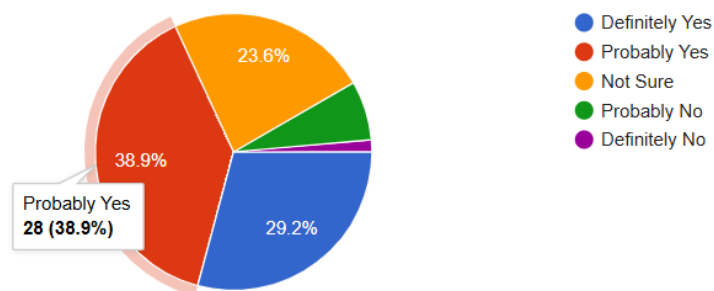
72 responses



18. Would you trust AI-generated summaries of official notices?

[Copy chart](#)

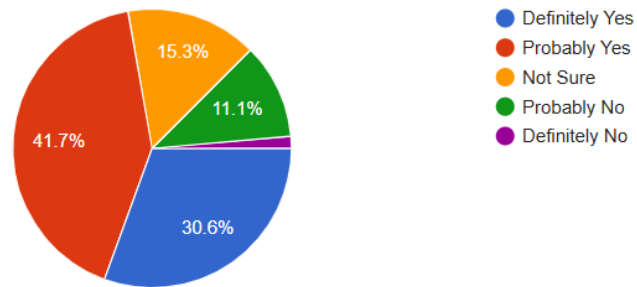
72 responses



19. Would you use an AI-powered cloud public notice management platform?

[Copy chart](#)

72 responses



20. Additional comments or suggestions about improving access to government public notices:

[Copy chart](#)

72 responses

◆ Summarize responses

